

单位代码： 10293 密 级：

南京邮电大学

专 业 学 位 硕 士 论 文



论文题目： 基于非负矩阵分解的人脸识别算法研究

学	号	<u>B20032222</u>
姓	名	<u>罗扬清</u>
导	师	<u>龙显忠</u>
专业学位类别		<u>计算机科学与技术</u>
类	型	<u>毕业论文</u>
专业（领域）		<u>计算机科学与技术</u>
论文提交日期		<u>2024年6月7日</u>

摘要

随着信息技术和人工智能的快速发展，人脸识别技术已广泛应用于安全监控、身份验证、人机交互等多个领域。然而，人脸识别技术在实际应用中仍面临诸多挑战，如光照变化、表情变化、遮挡等引起的识别精度下降问题。因此，研究高效且鲁棒性强的人脸识别算法具有重要的理论意义和实践价值。非负矩阵分解（Non-negative Matrix Factorization, NMF）是一种处理大规模高维数据的矩阵分解方法，它以分解的结果中不出现负值，提取的特征是基于部分的、局部化的、纯加性的描述等独特的优势区别于其它的方法。

本文深入研究了基于非负矩阵分解的人脸识别算法，并对其中的各种变体进行对比研究。通过在各种数据集上进行人脸识别实验，各种算法间的优劣形式以及鲁棒性可以清晰体现出来。

关键词：人脸识别，非负矩阵分解，特征提取，特征降维

Abstract

With the rapid development of information technology and artificial intelligence, face recognition technology has been widely applied in various fields such as security monitoring, identity verification, and human-computer interaction. However, face recognition technology still faces many challenges in practical applications, such as decreased recognition accuracy caused by changes in illumination, expression, and occlusion. Therefore, studying efficient and robust face recognition algorithms has significant theoretical and practical value. Non-negative Matrix Factorization (NMF) is a matrix decomposition method for processing large-scale high-dimensional data, which is distinguished from other methods by its unique advantages, such as the absence of negative values in the decomposition results and the extraction of features based on partial, localized, and purely additive descriptions. This article delves into the face recognition algorithm based on non-negative matrix factorization and conducts a comparative study of various variants. Through face recognition experiments on various datasets, the advantages and disadvantages of various algorithms can be clearly demonstrated.

Key Words: Face recognition, Non-negative Matrix Factorization, Feature Extraction, Feature Reduction

目 录

摘要	I
Abstract	II
插图索引	IV
表格索引	V
第一章 绪论	1
1.1 研究背景及意义	1
1.2 非负矩阵分解的发展与现状	1
1.3 基于 NMF 的人脸识别流程	2
1.4 论文结构安排	3
第二章 NMF 算法	6
2.1 实现原理	6
2.2 迭代过程	6
2.3 收敛性论证	7
第三章 NMF 的变体算法	12
3.1 LNMF 算法	12
3.1.1 实现原理	12
3.1.2 迭代过程	12
3.2 GNMF 算法	13
3.2.1 实现原理	13
3.2.2 迭代过程	13
3.3 GDNMF 算法	14
3.3.1 实现原理	14
3.3.2 迭代过程	15
第四章 分类器介绍	16
4.1 KNN 算法	16
4.2 SVM 算法	17
4.3 Adaboost 算法	19
4.4 对比与选择	20
第五章 结果分析	23
5.1 数据集介绍	23
5.2 实验设置	24
5.3 实验结果	25
5.3.1 ORL 数据集	27
5.3.2 UMIST 数据集	27
5.3.3 Yale 数据集	31
5.3.4 PIE 数据集	31
5.4 实验总结	35
第六章 鲁棒性测试	37
6.1 必要性分析	37
6.2 实验过程	37
6.3 结果分析	39

结论 43

参考文献 44

致谢 47

插图索引

图 1.1	人脸识别流程	3
图 1.2	使用 NMF 算法总步骤	4
图 1.3	人脸识别准确率计算步骤	5
图 4.1	KNN 算法流程图	18
图 4.2	SVM 算法流程图 ^[26,27]	19
图 4.3	Adaboost 算法流程图	20
图 5.1	ORL 数据集图例	23
图 5.2	PIE 数据集图例	23
图 5.3	UMIST 数据集图例	23
图 5.4	Yale 数据集图例	24
图 5.5	ORL 数据集的收敛情况	25
图 5.6	PIE 数据集的收敛情况	26
图 5.7	UMIST 数据集的收敛情况	26
图 5.8	Yale 数据集的收敛情况	27
图 5.9	ORL 数据集下各算法准确率折线图	28
图 5.10	ORL 数据集下各算法准确率折线图	28
图 5.11	ORL 数据集下各算法准确率折线图	29
图 5.12	UMIST 数据集下各算法准确率折线图	29
图 5.13	UMIST 数据集下各算法准确率折线图	30
图 5.14	UMIST 数据集下各算法准确率折线图	31
图 5.15	Yale 数据集下各算法准确率折线图	32
图 5.16	Yale 数据集下各算法准确率折线图	32
图 5.17	Yale 数据集下各算法准确率折线图	33
图 5.18	PIE 数据集下各算法准确率折线图	33
图 5.19	PIE 数据集下各算法准确率折线图	34
图 5.20	PIE 数据集下各算法准确率折线图	35
图 6.1	原图	38
图 6.2	椒盐噪声	38
图 6.3	高斯噪声	39
图 6.4	ORL 数据集添加椒盐噪声的收敛情况	40
图 6.5	ORL 数据集添加高斯噪声的收敛情况	40
图 6.6	Yale 数据集添加椒盐噪声的收敛情况	41
图 6.7	Yale 数据集添加高斯噪声的收敛情况	41
图 6.8	各算法在添加噪声后的 ORL 和 Yale 数据集下人脸识别准确率	42
图 6.9	各算法在添加噪声后的 PIE 和 UMIST 数据集下人脸识别准确率	42

表格索引

表 5.1 在 ORL 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 7：3） 27

表 5.2 在 ORL 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 8：2） 27

表 5.3 在 ORL 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 9：1） 28

表 5.4 在 UMIST 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 7：3） 29

表 5.5 在 UMIST 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 8：2） 30

表 5.6 在 UMIST 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 9：1） 30

表 5.7 在 Yale 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 7：3） 31

表 5.8 在 Yale 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 8：2） 31

表 5.9 在 Yale 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 9：1） 32

表 5.10 在 PIE 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 7：3） 33

表 5.11 在 PIE 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 8：2） 34

表 5.12 在 PIE 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 9：1） 34

第一章 绪论

1.1 研究背景及意义

数字图像处理是利用计算机算法对图像数据进行加工和分析的技术。它通常开始于将实际图像转换为数字数据，然后通过预处理来消除噪声和纠正误差。接下来包括图像分割来识别不同的区域或对象，以及特征提取来描述图像的边缘、色彩和纹理等属性。最后通过图像识别与解释来分类或识别图像内容。在某些情况下，还会进行图像重建，如对图像进行压缩和解压。

人脸识别是模式识别与图像处理学科中的核心研究主题，它既有深远的理论研究意义，也拥有极大的应用潜力。人脸识别主要由三大核心组成：人脸检测、人脸特征提取和人脸分类。人脸检测负责在输入的图像或视频帧中确认是否出现人脸，并确定其位置与尺寸；接着进入人脸特征提取阶段，从人脸图像中提炼出具有鉴别力的特征，这些特征对同一人来说是恒定不变的，但对不同人则各有差异；最终，在人脸分类阶段，系统会将提取出的特征与数据库中已有的人脸数据进行对比，从而识别和归类待鉴定的人脸。

在整个人脸识别过程中，人脸图像特征提取占据最重要位置。有效的特征提取方法能够提取出主要且具有代表性的人脸特征，使用这些特征将人脸数据归类为对应的模式类别，既简化了分类器设计，同时大大提升了人脸识别的准确率。非负矩阵分解 (Non-negative Matrix Factorization, NMF)^[1] 是一种处理大规模高维数据的矩阵分解方法，它以分解的结果中不出现负值，提取的特征是基于部分的、局部化的、纯加性的描述等独特的优势区别于其它的方法。作为一种新的特征提取方法，自从提出以后就被广泛应用于人脸识别领域。在人脸特征提取中，人们总是希望得到一个分解的结果更加稀疏化，局部特征更加明显，数据之间冗余更小，分解速度更快的矩阵分解方法。

1.2 非负矩阵分解的发展与现状

非负矩阵分解的发展始于 1999 年，由 Daniel D. Lee 和 H. Sebastian Seung 提出^[1]。这种矩阵分解方法以其分解后的所有分量均为非负值的特性，为数据分析和处理提供了一种新的视角。NMF 的非负性质使得其提取的特征具有加性和部分表示的特性，能够捕捉数据的本质结构。自提出以来，NMF 因其实现上的简便性、分解结果的可解释性以及存储空间的高效利用等优点，迅速成为信号处理、生物医学工程、模式识别、计算机视觉和图像工程等领域中备受关注的数据处理工具^[2]。随着研究的深入，研究者们提出了多种改进和优化算法，如约束非负矩阵分解、稀疏非负矩阵分解和图正则化非负矩阵分解等，进一步提高了 NMF 的性能

和应用范围。如今，基于 NMF 的方法已广泛应用于图像处理、文本分析、社交网络分析等多个领域，为科学研究和技术创新提供了有力的支持。

非负矩阵分解的几种变体算法扩展了原始 NMF 的适用范围和性能。

局部非负矩阵分解 (Local Non-negative Matrix Factorization, LNMF)^[3] 算法考虑了数据的局部结构信息，通过引入局部保持项来确保分解后的因子矩阵能够保留原始数据的局部特性。这对于处理具有明显局部结构特征的数据，如图像处理中的纹理分析，产生了极大的优化。

图正则化非负矩阵分解 (Graph regularized Non-negative Matrix Factorization, GNMF)^[4] 算法结合了图论和 NMF 的思想。它引入了图正则化项，利用数据的图结构信息来指导 NMF 的分解过程。通过构建数据点之间的图结构关系，GNMF 能够揭示数据中的潜在结构和关系，这在社交网络分析、生物信息学和图像处理等领域具有广泛的应用。

图正则判别非负矩阵分解 (Graph regularized Discriminative Non-negative Matrix Factorization, GDNMF)^[5] 算法是 GNMF 的进一步扩展，同时考虑了数据的判别性和图结构信息。在 GDNMF 中，除了图正则化项外，还引入了判别学习项，使得分解后的因子矩阵能够更好地区分不同类别的数据。这种结合使得 GDNMF 在处理具有类别标签的数据时表现出色，特别适用于分类和聚类任务。

这些变体算法在保持 NMF 基本思想的基础上，通过引入不同的约束项或度量方式，提高了 NMF 在不同应用中的适应性和性能。

1.3 基于 NMF 的人脸识别流程

本文所研究的非负矩阵分解算法是服务于人脸识别实验的，即非负矩阵分解是人脸识别算法的一部分。接下来会分别介绍人脸识别算法的宏观流程和非负矩阵分解算法的基本流程。

人脸识别的流程通常包括以下步骤：

首先，进行人脸图像采集及检测。使用摄像机或摄像头等设备捕获包含人脸的图像或视频流，并在采集到的图像中自动检测和跟踪人脸，确定人脸的位置和大小。这一步骤中，常用的算法有模板匹配模型、Adaboost 模型等，它们能够准确识别和定位人脸。

接下来，对人脸图像进行预处理。预处理包括一系列操作，如缩放、旋转、拉伸、光线补偿、灰度变换、直方图均衡化、规范化、几何校正、过滤以及锐化等。这些操作旨在改善图像质量，去除噪声和干扰，为后续的特征提取和识别提供更好的条件。

然后，进行人脸图像特征提取。特征提取是将人脸图像信息数字化的过程，将一张人脸图像转变为一串数字（特征向量）。常用的特征提取方法包括基于特征点间的欧氏距离、曲率、角度等提取特征向量，以及使用 DeepID 网络和卷积神经网络等深度学习模型进行特征提取。这些特征向量能够表征人脸的独特信息，为后续的匹配和识别提供基础。

在特征提取之后，可能还会进行人脸关键点及活体特征检测。关键点检测是根据人脸图像定位出人脸面部的关键区域，如嘴巴、鼻子、眼睛、耳朵、脸部轮廓等。这有助于更准确地识别和描述人脸特征。同时，活体检测通过眨眼、头部旋转、伸舌头等操作来判定是否为真人进行的操作，以防止使用照片或视频进行欺骗。

最后，进行匹配与识别。将提取出的人脸特征与数据库中存储的特征进行比对，以识别出该人脸的身份。这一步骤使用各种匹配算法和策略，如 K 最近邻（K-Nearest Neighbor, KNN）分类器、支持向量机（Support Vector Machine, SVM）等，以实现准确的人脸识别。

缩略流程如下图：

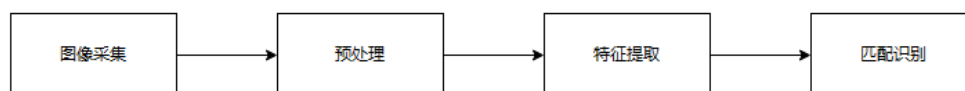


图 1.1 人脸识别流程

而本次毕设的目的是采用 NMF 进行人脸识别，所以对传统流程要进行部分改变。

非负矩阵分解是将非负约束整合到一般的矩阵分解中。其目标是找到两个非负矩阵，使其乘积能够最大可能地近似原矩阵。具体而言，非负矩阵分解是使用基向量的线性组合来表示原始数据，并且线性组合中的系数和基向量中的元素值都是非负的。这与大脑对于事物认知的方式是一致的，都是对于事物整体的认知是从对事物部分的认知开始。

NMF 算法的第一步是图像预处理。图像预处理包括两个方面，第一是将图片灰度化，其二是划分训练集和测试集。

随后通过训练集得出基矩阵，通过其对训练集和测试集分别降维。

最后一步则是指标计算，主要为记录损失函数以及进行人脸识别并计算准确率。具体流程见下图。

1.4 论文结构安排

本论文一共分为六章，主要目的是学习使用各种 NMF 算法在不同的数据集下进行人脸识别实验，并探究各算法间的优劣。

第一章绪论：简略地介绍毕业设计的目的以及人脸识别算法的流程。

第二章 NMF 算法：主要介绍 NMF 算法的原理以及实现方法。

第三章变体算法：主要介绍 NMF 算法的变体 LNMF、GNMF 与 GDNMF 算法的原理以及实现方法，并且与 NMF 算法进行对比。

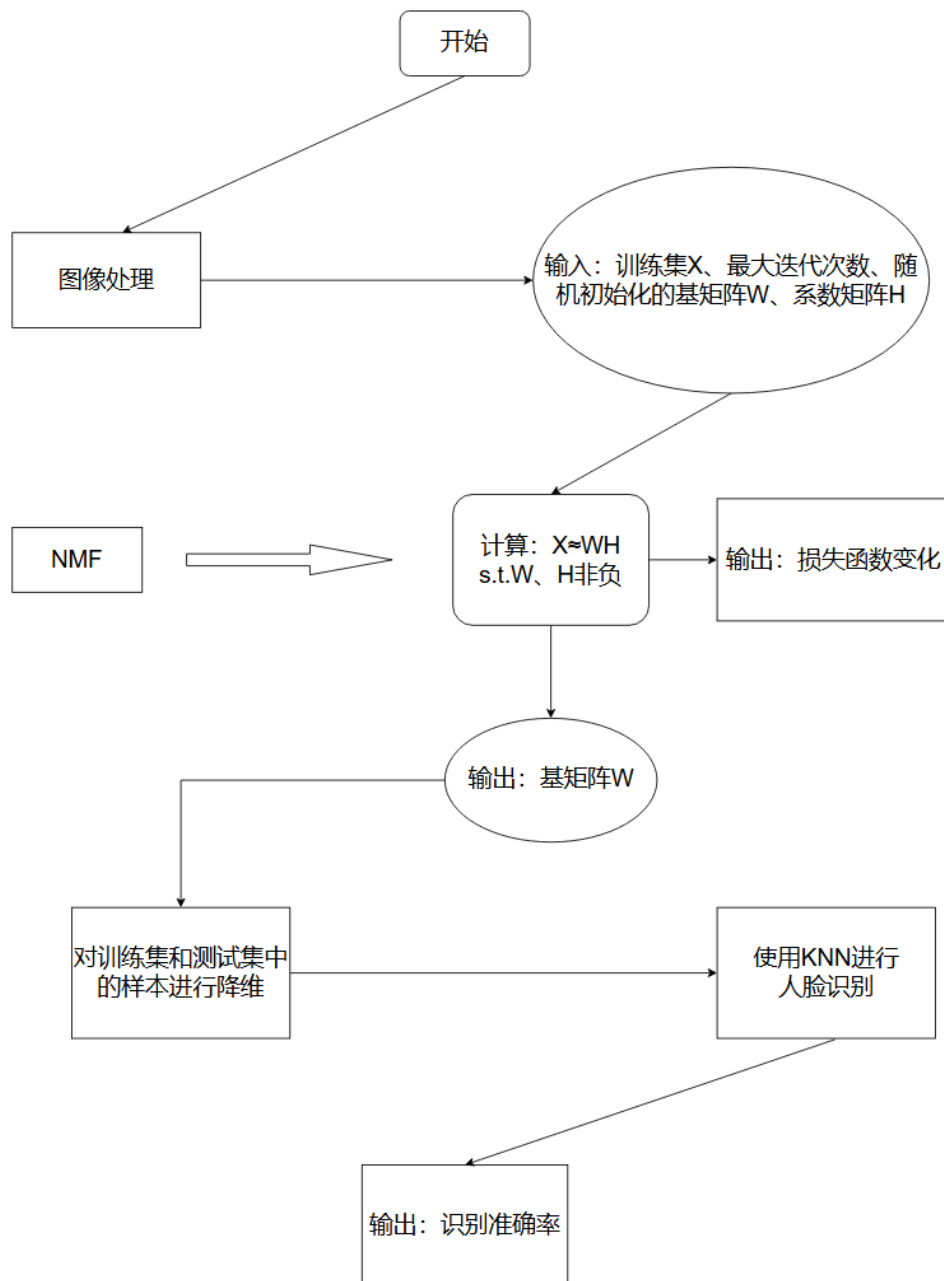


图 1.2 使用 NMF 算法总步骤

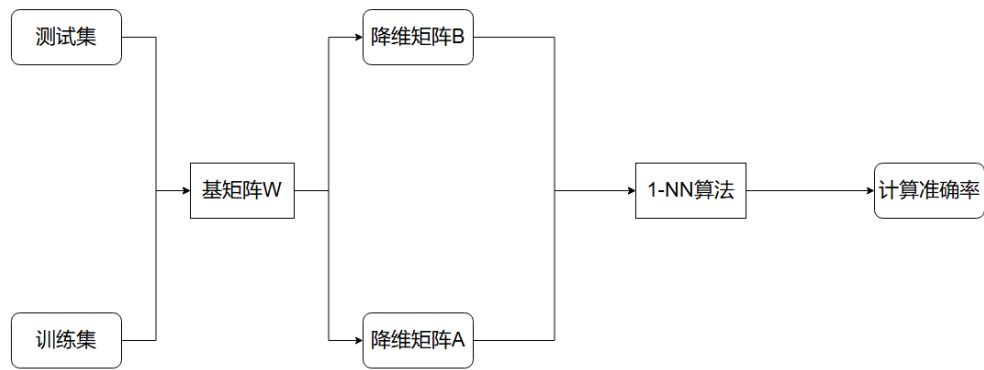


图 1.3 人脸识别准确率计算步骤

第四章分类器介绍：主要介绍在人脸识别中的主流分类器，分析其优劣，并解释选择 KNN 算法的原因。

第五章结果分析：凭借实验得出的结果对各个算法的准确性、收敛性进行分析，比较各个算法的优劣。

第六章鲁棒性的测试：构造噪声图像并借此检测各算法的鲁棒性。

第二章 NMF 算法

2.1 实现原理

非负矩阵分解是将非负约束整合到一般的矩阵分解中。其目标是找到两个非负矩阵，使其乘积能够最大可能地近似原矩阵。具体而言，非负矩阵分解是使用基向量的线性组合来表示原始数据，并且线性组合中的系数和基向量中的元素值都是非负的。这与大脑对于事物认知的方式是一致的，都是对于事物整体的认知是从对事物部分的认知开始^[1]。

非负矩阵分解是指对于任意给定的一个非负矩阵 $X \in R^{m \times n}$ ，能够将其分解成两个非负矩阵 $W \in R^{m \times r}$ 和 $H \in R^{r \times n}$ ($r \ll \min(m, n)$) 的乘积。NMF 分解后，基矩阵是每张人脸的一个特征部分，然后使用权重矩阵对这些特征进行加权处理得到一张人脸。

2.2 迭代过程

在使用 NMF 进行人脸识别的实验中，输入数据为包含人脸图片的数据集，这些数据集将在第四章进行详细介绍。数据集中的图片输入后，先对其进行灰度化处理，得到灰度矩阵 V_i ， i 为图片编号，并将其转化为列矩阵。然后将所有灰度列矩阵堆叠到一起，形成数据集 V 。然后按比例划分训练集 X 和测试集 Y ，训练集 X 的总数为比例 $\times$ 每类中所含图片，下取整，剩下的部分为测试集 Y 。

因为 X 本质上是灰度矩阵，所以矩阵中每个元素都是非负的，值在 0 到 255 之间； X 中的每一列代表一张图片，其列数代表数据集中图片的张数。我们的目标是找到两个非负矩阵 W 和 H ，使得它们的乘积近似等于原始矩阵 X ，即 $X \approx W \times H$ 。 W 为特征矩阵，最大限度地包含着数据集的特征， H 为系数矩阵。为了找到最优的 W 和 H ，需要定义一个目标函数来量化分解的好坏，我们将其称为损失函数。在本实验中，用基于 *Frobenius* 范数的损失函数：

$$F(H) = \frac{1}{2} \|X - WH\|_F^2 \quad (2-1)$$

并且使用乘性更新规则来进行迭代，迭代公示的推导以及损失函数收敛性的证明详见 2.3 小节。

迭代公式如下：

$$W_{ij} \leftarrow W_{ij} \times \frac{(XH^T)_{ij}}{(WHH^T)_{ij}} \quad (2-2)$$

$$H_{jk} \leftarrow H_{jk} \times \frac{(W^T X)_{jk}}{(W^T W H)_{jk}} \quad (2-3)$$

经过迭代所得出的 W 矩阵，本质上是一个数据集中共有的特征，而系数矩阵 H 则是判断图片类别的主要依据。

以 ORL 数据集为例，设训练集比例为 0.8。其中的图片格式为 32×32 ，共有 400 张图片。即每张图片需要用 32×32 的灰度矩阵来表示。将此矩阵转化为 1024×1 的列向量。再将 400 个列向量拼接成 1024×400 的矩阵 V ，经过留出法进行划分得到训练集 X 和测试集 Y 。 X 的大小为 1024×320 ， Y 的大小为 1024×80 。

将 X 进行迭代，由于迭代过程后损失函数一定收敛，所以先对 W 和 H 赋随机正值（值为 0~255 的正整数），然后对其按照公式进行迭代，记录损失函数的变化。经过迭代得出基矩阵 W ，计算投影矩阵 W^+ 。

$$W^+ = (W^T W)^{-1} W^T \quad (2-4)$$

最后使用投影矩阵 W^+ 分别对训练集 X 和测试集 Y 中的样本进行降维，并使用 KNN 算法计算准确率。至此一次最基本的算法至此完成。本项目将会将上述步骤重复进行，详见第五章。

2.3 收敛性论证

本文研究的 NMF 算法基于乘性更新规则进行迭代，本节的收敛性证明，主要是要证明在使用公式 (2-2) 和 (2-3) 对 W 和 H 分别进行迭代更新时，每次迭代更新后如果 $X - W \times H$ 的欧式平方距离递减，则说明说明重构误差函数是收敛的。

定理 1：如果满足条件

$$G(h, h') \geq F(h), G(h, h) = F(h) \quad (2.1)$$

则称 $G(h, h')$ 为 $F(h)$ 的一个辅助函数^{[1][6]}。

即给定变量 h' ， $G(h, h')$ 是关于变量 h 的函数。

引理 1：如果 G 是一个辅助函数，则 F 在

$$h^{t+1} = \arg \min_h G(h, h^t) \quad (2.2)$$

的迭代下为非增^{[1][7]}。

证明: $F(h^{t+1}) \leq G(h^{t+1}, h^t) \leq G(h^t, h^t) = F(h^t)$

此引理的含义为: 变量 h 迭代第 t 次的变量值为 h^t , 当我们在最小化 $G(h, h^t)$ 的时候, 目标函数值 F 也随之降低。

既然上面的定理以及引理已经提出, 则我们现在的目标是找到 NMF 损失函数的辅助函数。一旦辅助函数确定, 我们根据引理与定理就可以证明 NMF 的收敛性。

以下为辅助函数的构造过程:

引理 2: 如果 $K(h^t)$ 是对角矩阵且

$$K_{ab}(h^t) = \frac{\delta_{ab}(W^T W h^t)_a}{h_a^t} \quad (2-5)$$

则

$$G(h, h^t) = F(h^t) + (h - h^t)^T \nabla F(h^t) + \frac{1}{2}(h - h^t)^T K(h^t)(h - h^t) \quad (2-6)$$

是

$$F(h) = \frac{1}{2} \sum_i (x_i - \sum_a W_{ia} h_a)^2 \quad (2-7)$$

的一个辅助函数。

引理 2 的证明如下:

$$F(H) = \frac{1}{2} \|X - WH\|_F^2 \quad (2.3)$$

$$= \frac{1}{2} \sum_{j=1}^n \|X_{.j} - WH_{.j}\|_2^2 \quad (2-8)$$

$$= \sum_{j=1}^n \sum_{i=1}^d (X_{ij} - W_{i.} H_{.j})^2 \quad (2.4)$$

$$= \sum_{j=1}^n \sum_{i=1}^d (X_{ij} - \sum_{a=1}^c W_{ia} H_{aj})^2 \quad (2-9)$$

此处的 $F(H)$ 是关于 H 的函数, 在更新 H 的时候, 变量 W 为固定不变。2-8 就是将矩阵范式按列展开, 对应 n 列的 n 个子式是线性无关的, 所以最小化 $F(H)$ 就代表最小化这 n 个子式。以第 j 个子式为例写出新的目标式:

$$F(H_{.j}) = \frac{1}{2} \sum_{i=1}^d (V_{ij} - \sum_{a=1}^c W_{ia} H_{aj})^2 \quad (2-10)$$

定义 v, h 为 V 和 H 的第 j 列, 即 $V_{ij} = v_i, H_{aj} = h_a$, 所以上式简化为:

$$F(h) = \frac{1}{2} \sum_{i=1}^d (v_i - \sum_{a=1}^c W_{ia} h_a)^2 \quad (2-11)$$

至此 (2-7) 得证^{[2][8]}。

对 (2-7) 求导得:

$\nabla F(h) = \frac{\partial F(h)}{\partial h} = -W^T x + W^T W h$ (对向量求一阶导) $\frac{\partial^2 F(h)}{\partial h^2} = W^T W$ (对向量求二阶导, 得到 Hessian 矩阵)

$$\frac{\partial F(h)}{\partial h_a} = \sum_{i=1}^d -W_{ia} (x_i - \sum_{a=1}^c W_{ia} h_a) = -W_{.a}^T x + W_{.a}^T W h = -(W^T x)_a + (W^T W h)_a \quad (2.5)$$

$$\frac{\partial^2 F(h)}{\partial h_a^2} = \sum_{i=1}^d W_{ia} W_{ib} = W_{.a}^T W_{.b} \quad (2.6)$$

将目标式 $F(h)$ 二阶泰勒展开:

$$F(h) = F(h^t) + (h - h^t)^T \nabla F(h^t) + \frac{1}{2} (h - h^t)^T (W^T W) (h - h^t) \quad (2-12)$$

对比辅助函数 (2-6) 易得 $G(h, h^t) = F(h)$ 成立。所以只需证明 $G(h, h^t) \geq F(h)$, 即下式成立:

$$0 \leq (h - h^t)^T [K(h^t) - W^T W] (h - h^t) \quad (2-13)$$

所以我们需要证明 $(K(h^t) - W^T W)$ 半正定。我们可以借助下面的矩阵来证明:

$$M_{ab}(h^t) = h_a^t (K(h^t) - W^T W)_{ab} h_b^t \quad (2-14)$$

M_{ab} 是对矩阵 $(K(h^t) - W^T W)$ 中对应元素的缩放, 所以当矩阵 M 为半正定时, 矩阵 $(K(h^t) - W^T W)$ 也是半正定。

接下来证明 M 半正定:

$$x^t M x = \sum_{ab} X_a M_{ab} v_b \quad (2-15)$$

将引理 2 中的 $K_{ab}(h^t) = \delta_{ab}(W^T W h^t)_a / h_a^t$ 展开为

$$(W^T W h^t)_a = (W^T W)_a \cdot h^t = \sum_b (W^T W)_{ab} h_b^t \quad (2.7)$$

所以

$$= \sum_{ab} x_a h_a^t K_{ab}(h^t) h_b^t v_b - \sum_{ab} x_a h_a^t (W^T W)_{ab} h_b^t x_b \quad (2-15)$$

$$= \sum_{a,b,a=b} x_a h_a^t \frac{(W^T W h^t)_a}{h_a^t} h_a^t x_a - \sum_{ab} x_a h_a^t (W^T W)_{ab} h_b^t x_b \quad (2.8)$$

$$= \sum_a x_a \sum_b (W^T W)_{ab} h_b^t h_a^t x_a - \sum_{ab} x_a h_a^t (W^T W)_{ab} h_b^t x_b \quad (2.9)$$

$$= \sum_{ab} h_a^t (W^T W)_{ab} h_b^t x_a^2 - x_a h_a^t (W^T W)_{ab} h_b^t x_b \quad (2.10)$$

$$= \sum_{ab} (W^T W)_{ab} h_a^t h_b^t \left[\frac{1}{2} x_a^2 + \frac{1}{2} x_b^2 - x_a x_b \right] \quad (2.11)$$

$$= \frac{1}{2} \sum_{ab} (W^T W)_{ab} h_a^t h_b^t (x_a - x_b)^2 \quad (2.12)$$

$$\geq 0 \quad (2.13)$$

所以矩阵 M 半正定得证，矩阵 $(K(h^t) - W^T W)$ 半正定同样得证，可得引理 2 成立，即 $G(h, h^t)$ 为 $F(h)$ 的辅助函数。

我们可以通过最小化 $G(h, h^t)$ 来降低 $F(h)$ 的值。

令 G 对 h 的梯度为 0:

$$\frac{\partial G(h, h^t)}{\partial h} = \nabla F(h^t) + K(h^t)(h - h^t) = 0 \quad (2.14)$$

$$h^{t+1} \leftarrow h = h^t - K(h^t)^{-1} \nabla F(h^t) \quad (2.15)$$

对于每个元素有如下更新法则：

$$h_a^{t+1} = h_a^t - K_{aa}(h^t)^{-1} \frac{\partial F(h)}{\partial h_a} \quad (2.16)$$

$$= h_a^t - \frac{h_a^t}{(W^T W h^t)_a} (-(W^T x)_a + (W^T W h^t)_a) \quad (2.17)$$

$$= h_a^t \frac{(W^T x)_a}{(W^T W h^t)_a} \quad (2.18)$$

所以，关于 H 的乘性更新准则已经证明， W 的乘性更新准则可以类推。

至此，NMF 的收敛性已经得证。

第三章 NMF 的变体算法

3.1 LNMF 算法

局部非负矩阵分解是在非负矩阵分解的基础上，通过引入局部约束来提取图像的局部特征。与传统的 NMF 相比，LNMF 对基矩阵施加了空间局部性的限制，这使得在学习基矩阵的过程中，LNMF 能够更准确地捕捉图像中的局部结构和细节。因此，LNMF 在处理高维图像数据时，展现出了比 NMF 更加优异的性能。特别是随着原始图像维数的增加，LNMF 所学习到的特征更加局域化，这对于需要关注图像局部信息的任务（如人脸识别）来说，是非常有益的。

3.1.1 实现原理

LNMF 因为是基于 NMF 算法的变体算法，所以其实现与 NMF 类似，其中最大的不同是 NMF 采用的是基于 Frobenius 范数的损失函数（以本文为例），而 LNMF 算法则是利用 KL 散度进行优化，并通过引入局部约束^[9]使其更能把握图像的局部特征^[10]。其余步骤则与 NMF 类似。

3.1.2 迭代过程

因为 LNMF 算法与 NMF 算法类似，故不对算法的收敛性进行论述，其算法流程如下：

- （1）输入：训练样本矩阵 X ，测试样本矩阵 Y
- （2）设置分解矩阵的秩和迭代次数，将 X 分解为 W 和 H ， W 为特征矩阵， H 为系数矩阵。
- （3）利用 KL 散度量 LNMF 重构误差进行优化^[11]

$$\min_{W, H} D(X \| WH) + \alpha \sum_{i,j} U_{ij} - \beta \sum_{i,j} V_{ij} \quad (3.1)$$

其中参数 α 和 β 大于 0， $D(X \| WH)$ 表示 X 与 WH 间的 KL 散度， $U = W^T W$ ， $V = HH^T$

- （4）进行迭代

$$H_{kj} = \sqrt{H_{kj} \sum_i V_{ij} \frac{W_{ik}}{\sum_k W_{ik} H_{kj}}} \quad (3-1)$$

$$W_{kj} = \frac{W_{ik} \sum_j V_{ij} \frac{H_{kj}}{W_{ik} H_{kj}}}{\sum_{12} H_{kj}} \quad (3-2)$$

$$W_{ik} = \frac{W_{ik}}{\sum_i W_{ik}} \quad (3-3)$$

(5) 计算投影变换矩阵 W^+ :

$$W^+ = (W^T W)^{-1} W^T \quad (3.2)$$

(6) 将测试样本 Y 进行投影降维

(7) 计算准确率

3.2 GNMF 算法

基于流形学习理论, 研究人员提出了图正则非负矩阵分解模型。在该模型中, 高维空间中相邻的数据点在低维流形空间中仍然保持相邻^[12]。

3.2.1 实现原理

在 NMF 中, 一个非负的数据矩阵被分解为两个或多个非负矩阵的乘积, 从而揭示其潜在的结构和特征。这种方法在处理那些仅包含非负元素的数据矩阵时表现出了独特的优势^[13]。然而, 传统的 NMF 方法可能无法有效地保留数据中的局部结构信息, 这对于某些应用来说是非常重要的。

图正则非负矩阵分解通过在目标函数中加入图正则化项来解决这个问题。这个图正则化项通常基于数据点之间的相似性或距离来定义, 用于捕获数据中的局部结构信息。在优化过程中, 这个图正则化项会鼓励相似的数据点在分解后的特征空间中也保持相似^[14]。

具体来说, 图正则非负矩阵分解的目标函数通常包括两部分: 一部分是 NMF 的重构误差项, 用于确保分解后的矩阵能够很好地近似原始数据矩阵; 另一部分是图正则化项, 用于保留数据中的局部结构信息。通过优化这个目标函数, 我们可以找到一组非负矩阵, 它们不仅能够很好地表示原始数据, 还能够保留数据中的局部结构信息。

3.2.2 迭代过程

因为 GNMF 算法与 NMF 算法类似, 故不对算法的收敛性进行论述, 具体算法流程如下:

- (1) 输入: 训练样本矩阵 X , 测试样本矩阵 Y
- (2) 设置分解矩阵的秩和迭代次数, 将 X 分解为 W 和 H , W 为特征矩阵, H 为系数矩阵。
- (3) 利用欧氏距离度量 GNMF 重构误差并进行优化^[15]

$$\min_{W, H} \|X - WH\|_F^2 + \lambda \text{Tr}(HLH^T) \quad (3.3)$$

(4) 进行迭代

$$W_{ij} \leftarrow W_{ij} \times \frac{(XH^T)_{ij}}{(WH^TH)_{ij}} \quad (3-4)$$

$$H_{jk} \leftarrow H_{jk} \times \frac{(X^TW + \lambda RH)_{jk}}{(H^TWW + \lambda DH)_{jk}} \quad (3-5)$$

矩阵 R 为根据热核权重构建的权重矩阵, 对角矩阵 D 满足 $D_{ii} = \sum_j R_{ij}$, 拉普拉斯矩阵 $L = D - R$, Tr 为矩阵的迹, 正则化参数 $\lambda \geq 0$ 控制重构误差与正则项之间的平衡性。

(5) 计算投影变换矩阵 W^+ :

$$W^+ = (W^TW)^{-1}W^T \quad (3.4)$$

(6) 将测试样本 Y 进行投影降维

(7) 计算准确率

3.3 GDNMF 算法

在 GNMF 的目标函数中加入标签信息来构造图正则判别非负矩阵分解, 此模型能够使分类的精确度比 GNMF 更高^[5]。

3.3.1 实现原理

GDNMF 是一种结合了图结构和非负矩阵分解 (NMF) 的技术, 同时强调了数据的判别性。这种技术旨在从数据中提取出非负的特征表示, 并通过图结构来保留数据中的局部结构信息, 同时强调数据的判别能力^[18]。

在 GDNMF 中, 数据的判别性通常通过引入标签信息或者类别信息来实现。这意味着在分解过程中, 不仅要考虑数据的重构误差和图结构信息, 还要考虑数据的类别信息, 使得分解后的特征表示能够更好地区分不同的类别。

具体来说, GDNMF 的目标函数通常包括以下几个部分:

NMF 的重构误差项: 用于确保分解后的矩阵能够很好地近似原始数据矩阵。

图正则化项: 基于数据点之间的相似性或距离来定义^[18], 用于捕获数据中的局部结构信息。

判别项：利用标签信息或者类别信息来定义，用于强调数据的判别能力。这个判别项通常与数据的类别标签相关，鼓励相同类别的数据点在特征空间中的表示更加接近，而不同类别的数据点则更加远离。

通过优化这个目标函数，GDNMF 可以找到一组非负矩阵，它们不仅能够很好地表示原始数据，还能够保留数据中的局部结构信息，并且强调数据的判别能力。这使得 GDNMF 在分类、聚类任务中表现出色，因为它能够提取出对分类或聚类有用的特征表示。

3.3.2 迭代过程

因为 GDNMF 算法与 NMF 算法类似，故不对收敛性论证进行过多赘述，具体算法流程如下：

- (1) 输入：训练样本矩阵 X ，测试样本矩阵 Y
- (2) 设置分解矩阵的秩和迭代次数，将 X 分解为 W 和 H ， W 为特征矩阵， H 为系数矩阵。
- (3) 利用欧氏距离度量 GDNMF 重构误差并进行优化^[2]

$$\min_{W,H,A} \|X - WH\|_F^2 + \lambda \text{Tr}(HLH^T) + \beta \|Z - AH\|_F^2 \quad (3-6)$$

参数 λ 和 β 的值都是非负的， Z 是一个类别指示矩阵， A 是一个随机初始化的非负矩阵。

- (4) 参考 NMF 方法进行迭代

$$W_{ij} = W_{ij} \frac{(XH)_{ij}}{(WH^TH)_{ij}} \quad (3-7)$$

$$H_{ij} = H_{ij} \frac{(X^TW + \beta(H^-A^TA)^- + \beta(Y^TA)^+)_{ij}}{(HW^TW + \beta(HA^TA)^+ + \beta(Y^TA)^-)_{ij}} \quad (3-8)$$

- (5) 计算投影变换矩阵 W^+ ：

$$W^+ = (W^TW)^{-1}W^T \quad (3.5)$$

- (6) 将测试样本 Y 进行投影降维

- (7) 计算准确率

第四章 分类器介绍

在人脸识别中，分类器扮演着至关重要的角色，它是整个识别流程的核心组成部分。分类器的主要作用是对从人脸图像中提取的特征进行准确的分析和判断，以确定这些特征属于哪一个预设的类别或身份。

具体而言，当系统捕获到一个人脸图像后，会首先通过预处理步骤，如图像缩放，以确保后续特征提取的准确性。随后，特征提取算法会从预处理后的人脸图像中提取出具有代表性的特征信息，这些特征信息能够反映人脸的独特性。

一旦特征信息被提取出来，分类器就会对这些信息进行深入的分析 and 比较。分类器内部通常包含了一套复杂的算法和模型，这些算法和模型经过大量的训练和优化，已经学会了如何根据特征信息来区分不同的个体。在人脸识别中，分类器会将提取出的特征与已知人脸数据库中的特征进行比对，通过计算特征之间的相似度或距离，来确定待识别的人脸与哪个已知身份最为匹配。

总而言之，分类器在人脸识别中起到了一个“决策者”的角色。它不仅能够根据特征信息来准确判断人脸的身份，还能够处理复杂的情况，如人脸的角度变化、光照条件的变化等。通过分类器的精准判断，人脸识别系统能够在各种复杂场景下实现高效、准确的人脸识别功能。

4.1 KNN 算法

KNN 算法是一种基于实例的学习和懒惰学习的算法，其核心思想是通过查找数据集中与待分类样本最相似的 K 个样本，并根据这 K 个样本的类别来判断待分类样本的类别。KNN 算法简单易懂，无需训练，通过直接比较样本之间的相似度来进行分类或回归^[20]。

KNN 算法的发展历史可以追溯到 20 世纪 50 年代，当时人们开始探索基于实例的学习和局部逼近的概念。到了 1962 年，留日学者 Fix 提出了一个改进的基于实例的算法，这可以看作是 KNN 算法的初步形式。然而，KNN 算法的更广为人知和标准化的形式是由 Cover 和 Hart 在 1968 年正式提出的。他们详细阐述了 KNN 算法的核心思想，即通过查找数据集中与待分类样本最相似的 K 个样本，并根据这 K 个样本的类别来判断待分类样本的类别。这一思想奠定了 KNN 算法的基础，并为后来的研究提供了方向。

在 KNN 算法提出后的几十年里，人们对其进行了大量的研究和改进。这些改进主要集中在距离度量的选择、 K 值的选择、算法效率的优化以及处理不平衡数据等方面。最初，KNN 算法主要使用欧氏距离作为度量标准，但随着研究的深入，人们发现不同的距离度量方法可能对分类效果产生显著影响，因此研究者们开始探索各种距离度量方法。 K 值是 KNN 算法

中的一个重要参数，它的选择对分类效果有很大影响。为了找到最优的 K 值，研究者们提出了各种方法，如交叉验证、网格搜索等。此外，为了提高 KNN 算法的计算效率，研究者们提出了 KD-Tree、球树等数据结构来加速邻居的搜索过程，并基于核函数的 KNN 算法来处理非线性关系和数据分布不均匀的问题。在处理不平衡数据方面，研究者们提出了各种方法，如 SMOTE (Synthetic Minority Over-sampling Technique) 算法，来平衡不同类别的样本数量。

在 KNN 算法中，首先需要有一个带有标签的训练数据集，其中每个数据点都包含一组特征和一个与之对应的标签。然后，需要选择一个合适的 K 值，这个 K 值决定了在预测新数据点的标签时，要考虑的最近邻居的数量。 K 值的选择对算法的性能有重要影响，通常通过交叉验证或其他评估技术来确定最佳的 K 值^[21]。

当有一个新的、没有标签的数据点时，KNN 算法会计算这个新数据点与训练数据集中所有点之间的距离。距离的计算通常采用欧氏距离或曼哈顿距离等度量方式。然后，算法会选择距离新数据点最近的 K 个数据点作为邻居。最后，根据这 K 个邻居的标签来预测新数据点的标签。在分类问题中，这通常是通过多数投票来实现的，即选择出现次数最多的类别作为预测结果；在回归问题中，则通常是取这 K 个邻居的标签的平均值作为预测结果。

KNN 算法的优点在于其简单易懂、易于实现，并且无需估计参数，也无需训练，因此可以快速地对新数据进行预测。此外，KNN 算法对数据分布没有假设性，适用于非线性数据。然而，KNN 算法也存在一些缺点。首先，当数据集很大时，计算新数据点与所有训练数据点之间的距离会导致计算量较大，预测速度变慢^[22]。其次，KNN 算法对 K 值的选择非常敏感，如果 K 值选择不当，可能会导致预测结果不准确。此外，KNN 算法对异常值也较为敏感，因为异常值可能会成为某些数据点的最近邻居，从而影响预测结果。

在实际应用中，KNN 算法有着广泛的应用场景。例如，在个性化推荐系统中，可以通过计算用户之间的相似度，找到相似用户，并根据相似用户的行为给用户提供个性化的推荐。在文本分类中，KNN 算法可以通过计算文本数据之间的相似度，将未分类的文本分为不同的类别^[23]，这在垃圾邮件过滤、情感分析等任务中非常有用。此外，KNN 算法还可以应用于图像识别领域，通过计算图像之间的相似度，帮助识别图像中的物体、人脸等信息。这些应用场景展示了 KNN 算法在解决实际问题时的有效性和实用性。

KNN 算法的流程图如下：

4.2 SVM 算法

SVM 算法是一种在机器学习领域广泛应用的算法，其历史可以追溯到上世纪六十年代，但真正的突破和广泛应用是在上世纪九十年代初期。最初，SVM 主要关注于线性分类问题，试图在数据中找到一个最优的超平面来划分不同类别的数据点。这一阶段的 SVM 被称为线性支持向量机，奠定了 SVM 算法的基础。

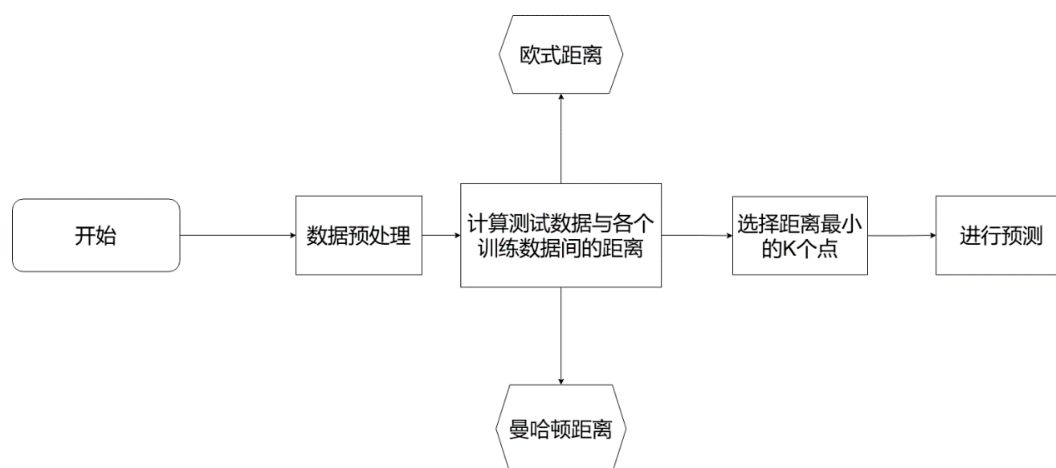


图 4.1 KNN 算法流程图

然而，随着研究的深入，研究者们发现许多实际问题中的数据并不是线性可分的。为了解决这一问题，SVM 引入了“软间隔”的概念，允许一些样本点被错误地分类，从而提高了算法的泛化能力。这种扩展后的 SVM 被称为线性不可分支持向量机，它进一步扩展了 SVM 算法的应用范围。

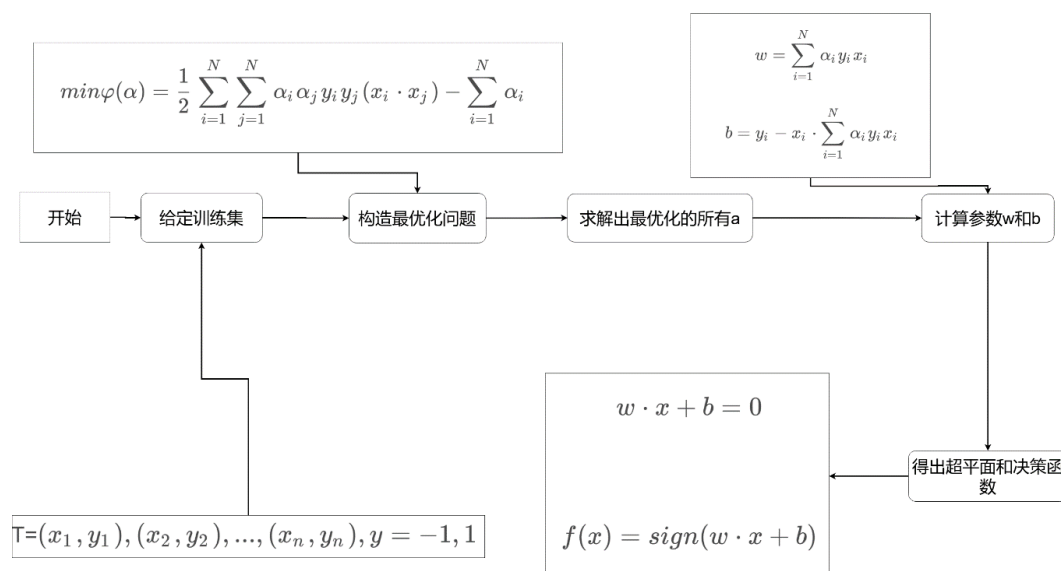
随着研究的进一步深入，研究者们发现对于很多实际问题，数据之间的关系并不是线性的，而是非线性的。为了处理这类问题，SVM 引入了“核函数”的方法。核函数能够将原始数据映射到更高维的空间，使得原本非线性可分的数据在高维空间中变得线性可分。这一阶段的 SVM 算法被称为非线性支持向量机，也是现代意义上的 SVM 算法^[24]。

随着 SVM 算法的广泛应用，研究者们也在不断地对其进行改进。这些改进算法在寻找最佳分类直线（或超平面）时采用了各种优化策略，使得 SVM 能够更好地适应不同的数据集和任务需求。

SVM 算法的应用领域也逐渐扩展到了多分类、回归分析和异常检测等领域。在文本分类和情感分析领域，SVM 可以通过提取文本特征并进行分类来识别文本的主题或情感倾向^[25]；在图像识别和人脸检测领域，SVM 可以通过提取图像特征并进行分类来实现高效的图像识别和人脸检测；在生物信息学和医学诊断领域，SVM 可以通过分析基因序列或医学图像来辅助医生进行疾病诊断和治疗方案制定。

总的来说，SVM 算法的发展历程是一个不断发展和完善的过程。从最初的线性分类器到现代的非线性支持向量机，再到各种改进算法的出现，SVM 算法不断适应着新的数据需求和应用场景，成为机器学习领域中一种重要的分类和回归分析工具。

SVM 的流程图如下：

图 4.2 SVM 算法流程图^[26,27]

4.3 Adaboost 算法

Adaboost 算法的发展历史可以追溯到对 Boosting 算法的探索和改进。Boosting 算法，作为集成学习的一个重要分支，其核心思想是通过迭代地组合多个弱学习器（分类器或回归器），以形成一个强大的集成学习器，即强学习器。这一思路的初衷是，即使单个学习器的性能有限，但通过合理的方式将它们结合起来，也可能达到或超过单一高性能学习器的效果^[28]。

然而，早期的 Boosting 算法存在一些问题，例如需要提前确定弱学习器识别准确率的下限，这在实际应用中往往难以实现。为了克服这些限制，Yoav Freund 和 Robert Schapire 在 1995 年提出了 Adaboost 算法。Adaboost 算法是一种自适应的 Boosting 算法，它引入了样本权重的概念，并在每次迭代中根据弱学习器的性能调整这些权重。

具体来说，Adaboost 算法在开始时为所有训练样本分配相同的权重。然后，在每一次迭代中，它训练一个弱学习器，并计算该学习器在加权训练样本上的错误率。根据这个错误率，Adaboost 算法会更新样本的权重，增加错误分类样本的权重，并减少正确分类样本的权重。这样，在下一次迭代中，弱学习器将更加关注那些在前一次迭代中错误分类的样本。

通过这种迭代过程，Adaboost 算法能够逐步“聚焦”于那些难以分类的样本，从而提高整个集成学习器的性能。此外，Adaboost 算法还根据每个弱学习器的性能（即错误率）来赋予它们不同的权重。具体来说，错误率较低的弱学习器在最终决策中将拥有更大的影响力^[29]。

Adaboost 算法的特点主要体现在以下几个方面：

首先，它的自适应性是其最显著的特点之一。通过调整样本权重和弱学习器权重，Adaboost 算法能够根据不同数据集和弱学习器的特性来优化整个集成学习器的性能。这种自适应性使得 Adaboost 算法在实际应用中具有很强的灵活性和通用性。

其次，Adaboost 算法具有较强的鲁棒性。由于它采用了集成学习的思想^[30]，将多个弱学

习器的结果进行综合，因此即使某个弱学习器出现错误，也不会对整个集成学习器的性能产生太大影响。这种鲁棒性使得 Adaboost 算法在处理复杂和不确定的数据集时表现出色。

此外，Adaboost 算法还具有高效性和泛化能力强的特点。它的迭代过程能够快速收敛到最优解，从而提高了算法的训练效率。同时，由于它关注于难以分类的样本并赋予不同的权重给不同的弱学习器，这使得 Adaboost 算法在面对新的、未见过的数据时也能够表现出较好的性能。

总的来说，Adaboost 算法是一种强大而灵活的集成学习算法，它通过自适应地调整样本权重和弱学习器权重来提高整个集成学习器的性能。由于其具有自适应性、鲁棒性、高效性和泛化能力强等特点，Adaboost 算法在机器学习领域得到了广泛的应用，特别是在分类和回归问题中表现出色。

Adaboost 算法的流程图如下：

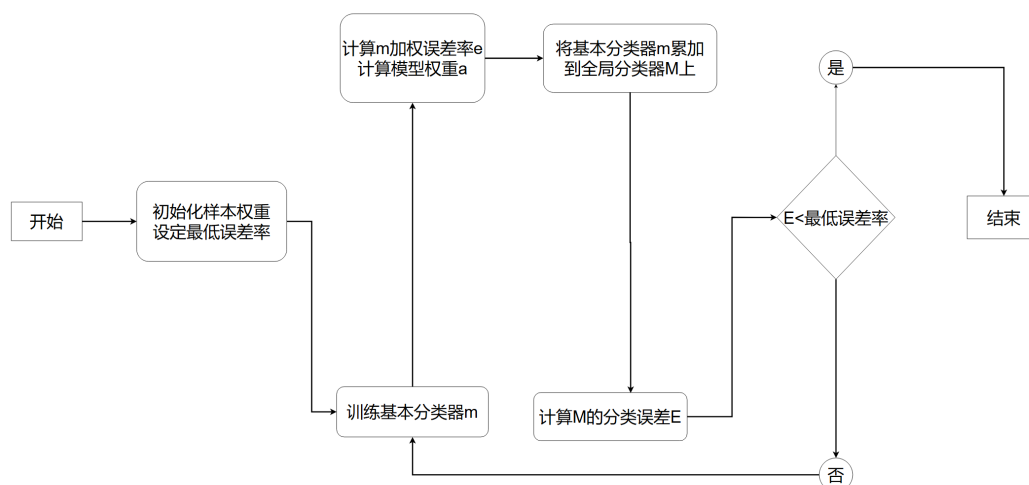


图 4.3 Adaboost 算法流程图

4.4 对比与选择

KNN、SVM 和 AdaBoost 各自具有独特的优点和缺点，适用于不同的场景和问题。以下是对这三种算法的比较：

KNN:

优点:

简单易懂: KNN 算法基于实例的学习，其原理直观易懂，易于实现。

适用于多类别问题: KNN 可以处理多类别的分类问题，并且在类别之间没有明显界限时也有较好的效果。

无假设性: KNN 对数据分布没有假设性，适用于非线性数据。

适用于大型数据集：KNN 的训练时间复杂度较低，可以处理大型数据集。

可在线学习：当新的样本出现时，可以直接将其加入已有的训练样本中进行分类。

缺点：

计算复杂度高：在预测时需要计算待分类样本与所有训练样本的距离，当数据集较大时计算复杂度较高。

需要确定 K 的值：K 值的选择对结果有重要影响，选择不恰当的 K 值可能会产生较大的误差。

对异常值敏感：异常值可能会对分类结果产生较大影响。

数据不平衡问题：当数据集中某个类别的样本数量较少时，KNN 的分类结果可能会受到影响。

SVM:

优点：

在处理高维数据和样本量较小的情况下表现良好，具有较强的泛化能力。

通过使用核函数，SVM 可以解决非线性分类问题。

通过间隔最大化，SVM 对噪声数据具有较好的鲁棒性。

缺点：

在处理大规模数据时计算复杂度较高。

对于大规模数据集，训练时间较长。

对参数的选择和核函数的选择比较敏感，需要进行调优。

AdaBoost:

优点：

能够自动调整弱学习器的权重，使得错误率低的弱学习器在最终决策中具有更大的影响力。

适用于各种弱学习器，包括决策树、神经网络等。

对噪声数据和异常值具有一定的鲁棒性。

缺点：

如果某些类别的样本数量较少，可能会导致过拟合。

在某些情况下，可能会受到弱学习器性能的限制。

在本次毕业设计中，采用的数据集为小型数据集，但是需要进行循环计算以得出结果，即每次循环都必须重新进行人脸识别，所以每次进行人脸识别的训练集以及测试集都不同。所以为了节约计算资源以及保证准确度，对于需要根据数据集来进行调参的 SVM 算法就不予考虑。

由于数据集较小，最大的数据集也不超过 2000 张图片，且同一类别中的图片不超过 20 张，所以 Adaboost 算法可能会出现过拟合的情况。

本次毕业设计重点在于讨论非负矩阵算法，所以计算资源主要侧重于此方面，且数据集较小，所以采用 KNN 算法的子算法 1NN 算法（1-Nearest Neighbor）进行人脸识别。

1NN 算法是 KNN 算法的一个特例，其中 K 的值被设定为 1。这意味着在分类或回归任务中，1NN 算法仅考虑与待分类或预测样本最近的一个邻居的类别或值。

在 1NN 算法中，对于给定的待分类或预测样本，首先计算该样本与训练数据集中所有样本之间的距离。然后，找出距离待分类或预测样本最近的那一个样本，即“最近邻”。最后，将待分类或预测样本的类别或值设为最近邻的类别或值。

1NN 算法的优点在于其简单直观，易于理解和实现。由于只考虑一个最近邻，因此计算量相对较小，预测速度较快。

鉴于此，采用 1NN 算法进行人脸识别较为合适。

第五章 结果分析

5.1 数据集介绍

ORL 数据集，也被称为 ORL 人脸数据集或 Olivetti Research Laboratory 人脸数据库，是由英国剑桥的 Olivetti 研究实验室在 1992 年 4 月至 1994 年 4 月期间创建的一个标准人脸数据库。该数据集包含了 40 个不同人的 400 张图像，每个人的图像被组织在一个单独的目录下，每个目录包含 10 张图像。这些图像是在不同的时间、不同的光照、不同的面部表情（如睁眼/闭眼、微笑/不微笑）和面部细节（如戴眼镜/不戴眼镜）环境下采集的。所有的图像都是在较暗的均匀背景下拍摄的，主要拍摄的是正脸（有些带有略微的侧偏）。图像以 PGM 格式存储，为灰度图，图像大小宽度为 92 像素，高度为 112 像素。本次所采用的 ORL 数据集每类共 10 张图片，经过剪裁，大小均为 32*32 像素。



图 5.1 ORL 数据集图例

PIE 数据集包含 68 个不同人的 41,368 张图像，每个人有 13 种不同的姿势、43 种不同的照明条件和 4 种不同的表情。在本实验中，使用的是 PIE 数据集中姿势相同但是光照条件不同的图像，其中共 68 类人脸，每类人脸有 21 张图像，经过剪裁，大小均为 32*32 像素。



图 5.2 PIE 数据集图例

UMIST 图像集由英国曼彻斯特大学建立。包括 20 个不同的人共 564 幅图像，每个人具有不同角度、不同姿态的多幅图像。本次所采用的 UMIST 数据集每个类别的图像数量从 19 到 48 不等，同时包含了不同程度上的侧脸影响，经过剪裁，大小均为 40*40 像素。



图 5.3 UMIST 数据集图例

Yale 数据集是计算机视觉领域中的一个重要的人脸识别数据集，由耶鲁大学计算视觉与控制中心创建。该数据集包含 15 位志愿者的图像，每位志愿者都有不同表情、姿态和光照下的多张人脸图像。具体来说，Yale 人脸数据库包含 165 张图片，本次所采用的 Yale 数据集同一类包含 9 到 11 张图片，经过剪裁，大小均为 40*40 像素。



图 5.4 Yale 数据集图例

5.2 实验设置

本次实验是在 MATLAB 平台进行的，共采用了 4 个数据集。

MATLAB 是 MathWorks 公司开发的一款强大的商业数学软件，集成了数值计算、数据分析、可视化以及算法开发等多种功能。它以矩阵为基本数据单位，提供了与数学和工程问题相似的表达式和命令，使得数值计算更为简便。此外，MATLAB 拥有丰富的工具箱，涵盖复杂系统仿真、信号处理、优化、神经网络等多个领域，为用户提供了强大的支持。其直观易用的编程环境和与其他编程语言的连接能力，使得 MATLAB 成为科学计算、工程设计以及数值计算领域的全面解决方案。

4 个数据集均经过剪裁与灰度化，并存储为 .mat 文件。各数据集的统计特性在上文已经介绍，在此处就不再赘述。与经过灰度化的图片一起存储的还有类别向量，大小为图片个数 * 1。本向量的作用是对图片进行分类以便区分图片所属的人，用以划分训练集和降维处理后进行人脸识别。

以 ORL 数据集为例，读取的 .mat 文件主要包含两个矩阵：fea 矩阵和 gnd 向量。fea 矩阵储存图片信息，大小为 1024*400。400 为图片张数，1024 为图片转化为灰度矩阵后所包含的像素（1024=32*32）。gnd 向量大小为 400*1，400 为图像张数。此列向量的值的范围为 1~40（40 为数据集所包含的不同人的类别总数）。因为此数据集每人有 10 张图片，故 gnd 向量的 1~10 行的值为 1，以此类推。

读取数据集后，首要任务是划分训练集与测试集。本次使用留出法进行划分，训练集与测试集的比例并不固定，分别为 7: 3，8: 2，9: 1，并向下取整。重复数次分别实验以保证严谨性。

由于本次实验所研究的对象为 NMF 及其各种变体算法，所以接下来对参数的介绍将分为共同参数以及独有参数。

数据集 V 使用留出法进行划分，分为训练集 X 以及测试集 Y。

矩阵 W 为特征矩阵，矩阵 H 为系数矩阵，算法的目的是找到两个非负矩阵 W 和 H，使得它们的乘积近似等于原始矩阵 X，即 $X \approx W \times H$ 。i,j,k 作为矩阵下标使用，例如 W_{ij} 表示矩阵 W 第 i 行第 j 列的元素。

LNMF 算法中，参数 α 和 β 为非负，作为可调参数使用，进行调参后可以提高算法收敛的速度。 $D(X||WH)$ 表示 KL 散度， $U = W^T W, V = H H^T$ 。

GNMF 算法中，矩阵 R 为根据热核权重构建的权重矩阵，对角矩阵 D 满足 $D_{ii} = \sum_j R_{ij}$

, 拉普拉斯矩阵 $L = D - R$, Tr 为矩阵的迹, 正则化参数 $\lambda \geq 0$ 控制重构误差与正则项之间的平衡性。实验中通过调整 λ 的值可以提高算法收敛的速度。

GDNMF 算法中, 参数 λ 和 β 的值都是非负的, 进行调参后可以提高算法收敛的速度。Z 是一个类别指示矩阵, A 是一个随机初始化的非负矩阵。

5.3 实验结果

本次实验为对比实验, 将在四个数据集上反复进行实验以验证各项参数对于识别准确率的影响。具体过程如下: 迭代次数分别取 500, 1000, 2000, 3000 次, 训练集与测试集的划分比例分别为 9:1, 8:2, 7:3, 降维维数分别取 5, 6, 7, 8, 9, 10 维进行循环实验。每次实验均随机取训练集和测试集 5 次, 分别进行实验以保证严谨性。下面是实验结果以及分析。

以下图片为在四个数据集上各算法的损失函数的收敛情况。由图易得, 当迭代次数在 1000 时, 损失函数的变化率趋于稳定, 当迭代次数为 3000 时, 变化率趋近于 0, 此时可以认为算法收敛。

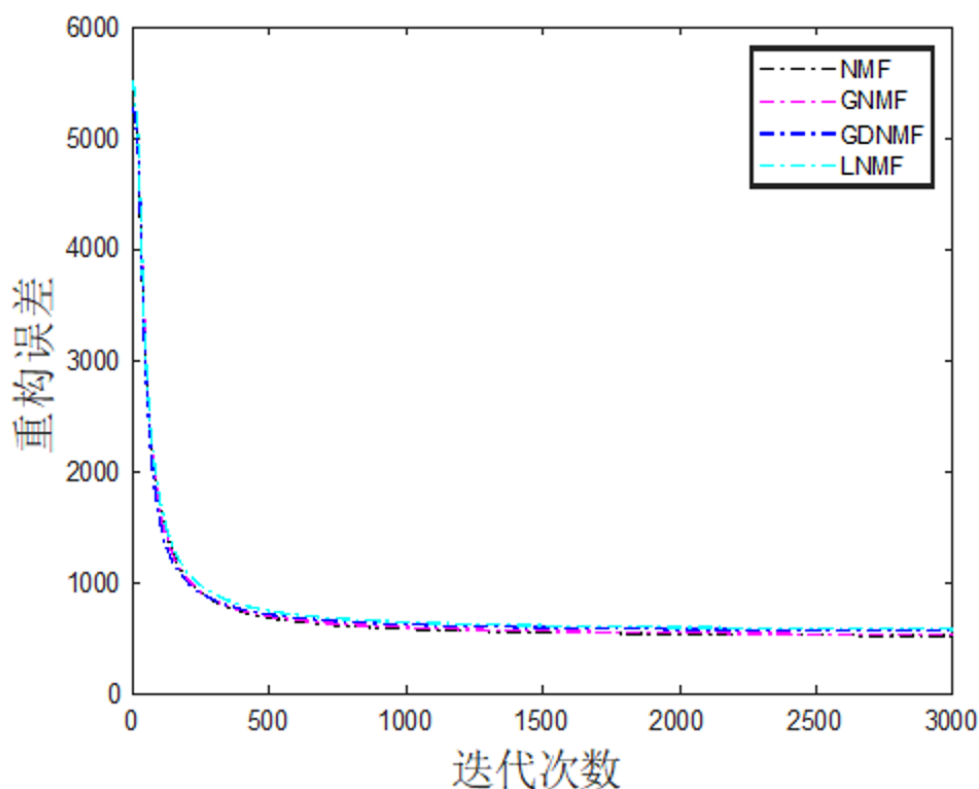


图 5.5 ORL 数据集的收敛情况

由于各算法在迭代次数为 3000 时收敛, 所以接下来的结果分析将聚焦在训练集的占比和降维维数对于算法准确率的影响。

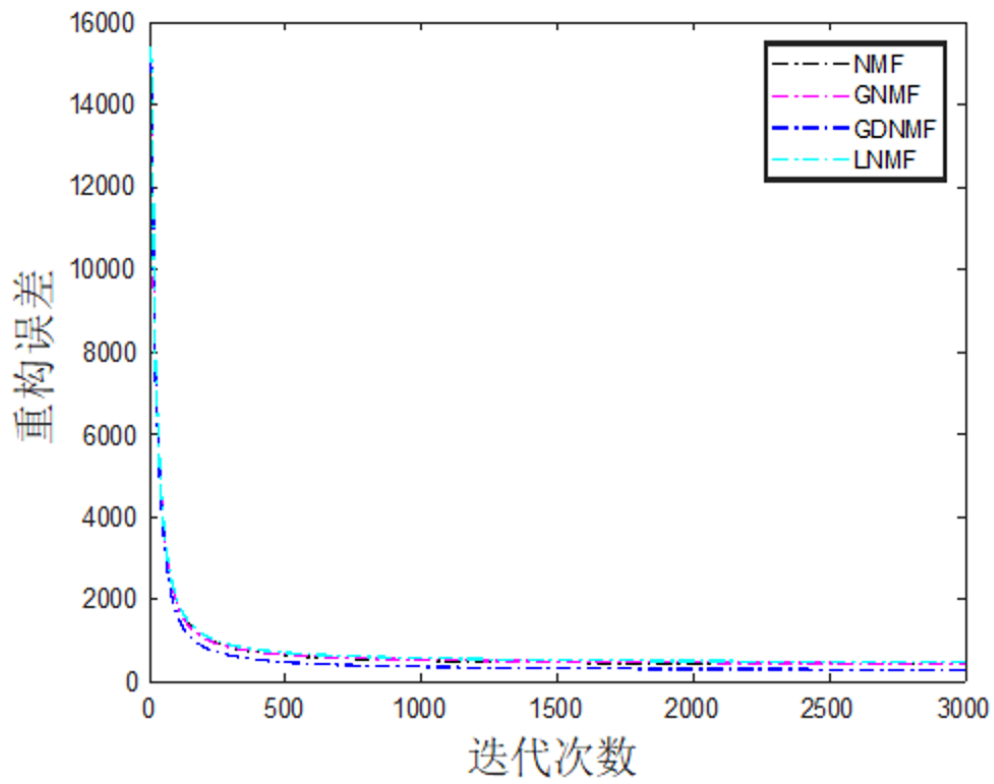


图 5.6 PIE 数据集的收敛情况

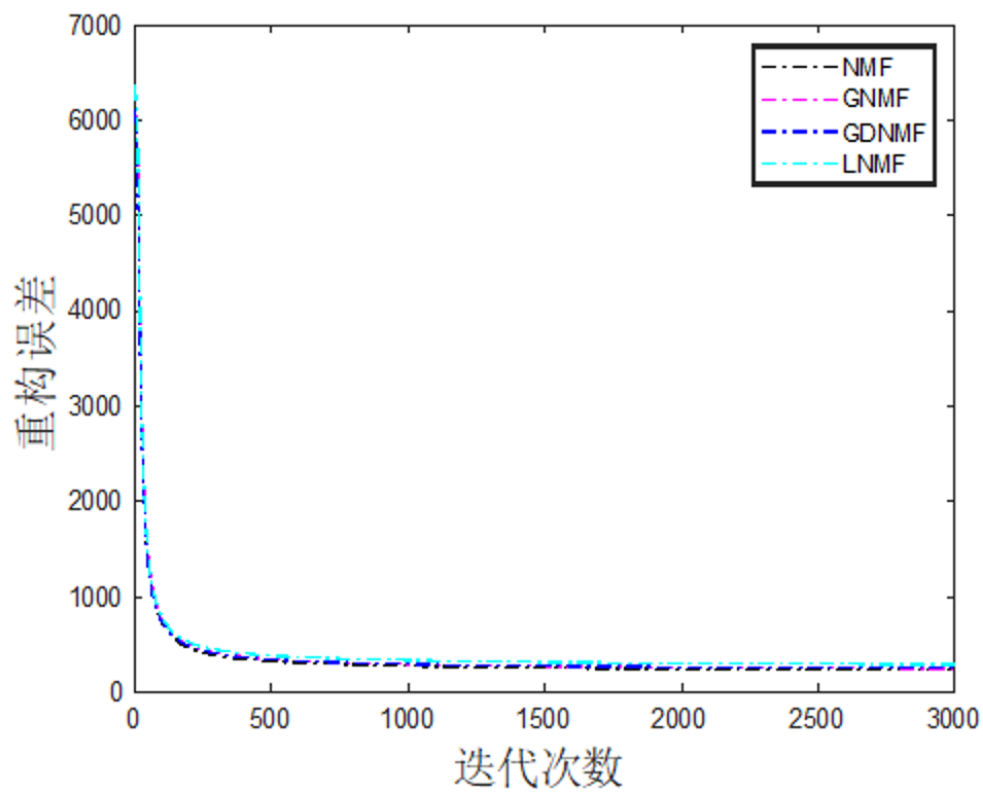


图 5.7 UMIST 数据集的收敛情况

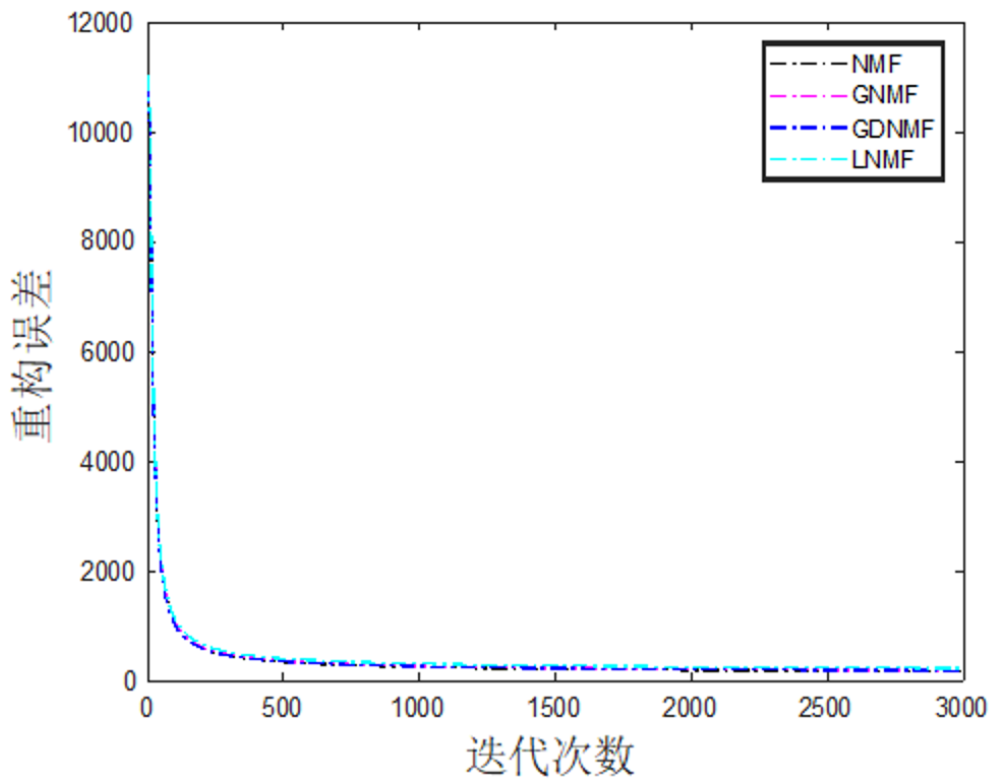


图 5.8 Yale 数据集的收敛情况

5.3.1 ORL 数据集

经过迭代后，各算法进行人脸识别的准确率如下：

表 5.1 在 ORL 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 7：3）

维数算法	5	6	7	8	9	10
NMF	0.9133	0.9033	0.9217	0.9000	0.8967	0.9017
LNMF	0.9150	0.9100	0.9233	0.9100	0.9050	0.9150
GNMF	0.9067	0.9083	0.9183	0.9067	0.8933	0.9033
GDNMF	0.9217	0.9217	0.9100	0.8950	0.8900	0.8983

表 5.2 在 ORL 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 8：2）

维数算法	5	6	7	8	9	10
NMF	0.9025	0.8925	0.9375	0.9250	0.9300	0.9125
LNMF	0.9050	0.9100	0.9525	0.9350	0.9300	0.9500
GNMF	0.8975	0.8925	0.9400	0.9350	0.9300	0.9175
GDNMF	0.9375	0.8925	0.9275	0.9325	0.9200	0.8925

5.3.2 UMIST 数据集

经过迭代后，各算法进行人脸识别的准确率如下：

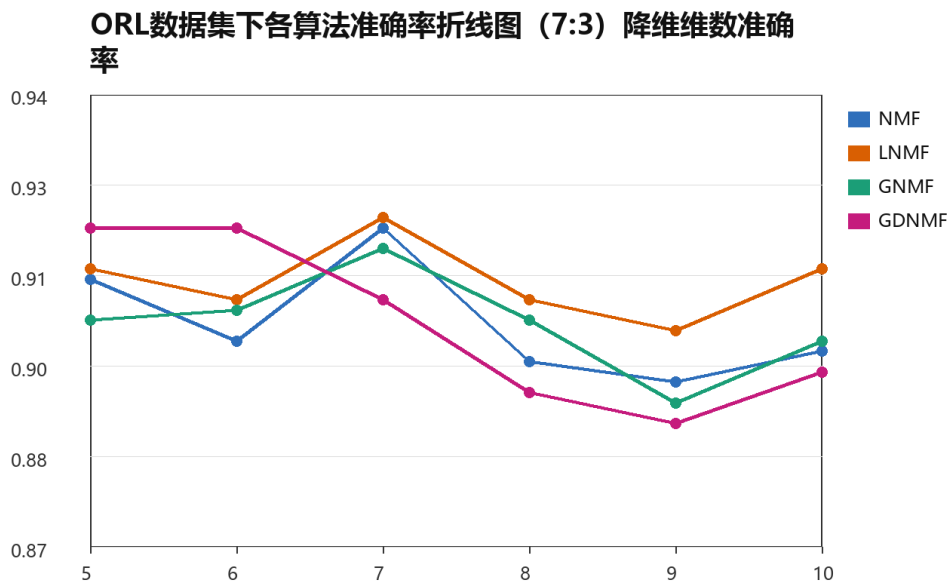


图 5.9 ORL 数据集下各算法准确率折线图

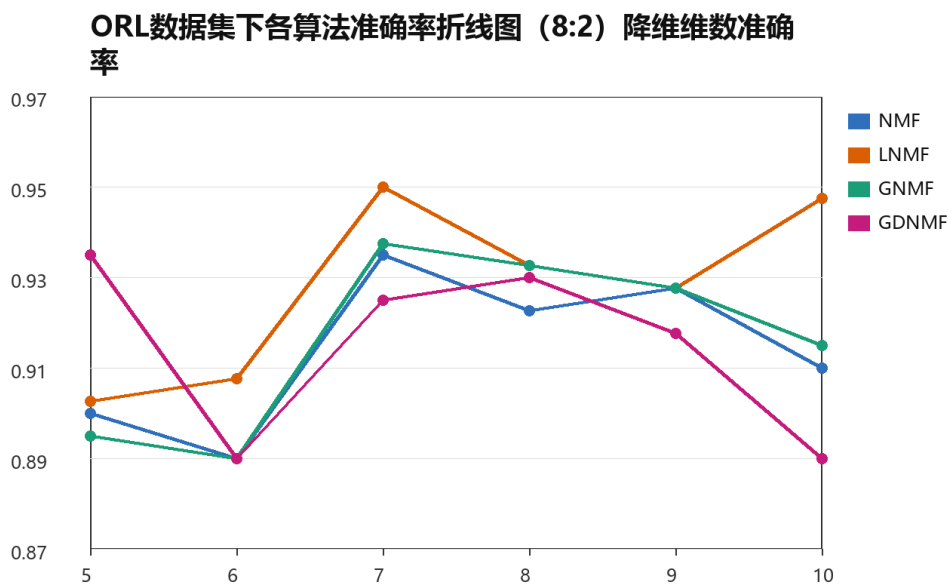


图 5.10 ORL 数据集下各算法准确率折线图

表 5.3 在 ORL 数据集下各算法进行人脸识别的准确率 (训练集: 测试集 = 9: 1)

维数算法	5	6	7	8	9	10
NMF	0.9100	0.9250	0.9400	0.9650	0.9250	0.9650
LNMf	0.9350	0.9500	0.9500	0.9350	0.9750	0.9700
GNMF	0.9300	0.9300	0.9250	0.9600	0.9350	0.9500
GDNMF	0.9350	0.9400	0.9200	0.9800	0.9250	0.9400

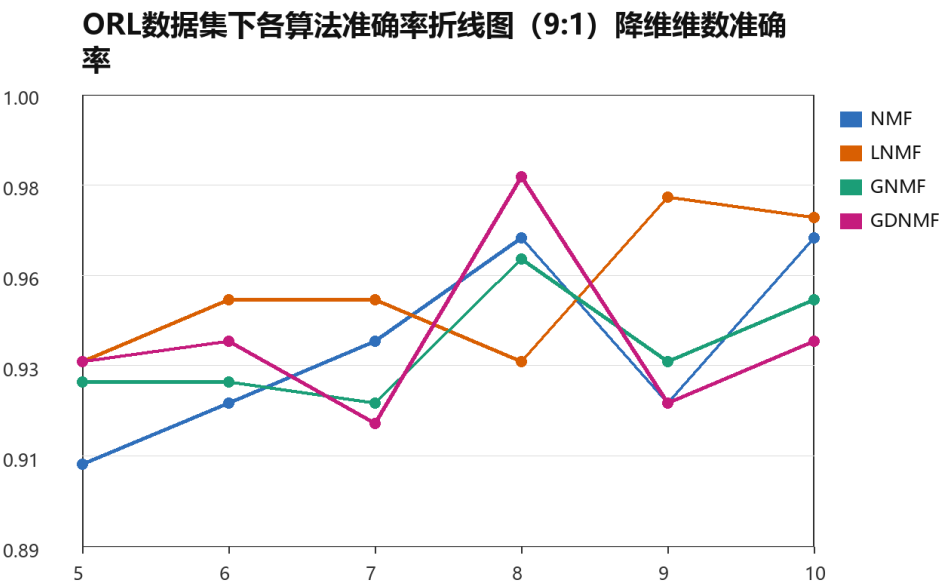


图 5.11 ORL 数据集下各算法准确率折线图

表 5.4 在 UMIST 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 7：3）

维数算法	5	6	7	8	9	10
NMF	0.8634	0.8639	0.8520	0.8474	0.8317	0.8115
LNMF	0.9002	0.9062	0.8869	0.9163	0.8984	0.8910
GNMF	0.8602	0.8680	0.8584	0.8529	0.8437	0.8372
GDNMF	0.8759	0.8736	0.8543	0.8547	0.8446	0.8248

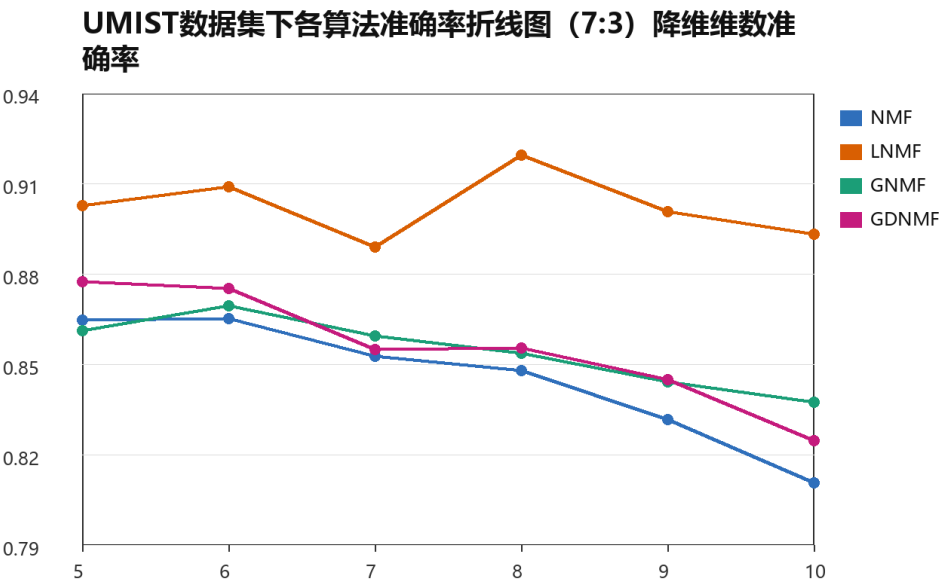


图 5.12 UMIST 数据集下各算法准确率折线图

表 5.5 在 UMIST 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 8：2）

维数算法	5	6	7	8	9	10
NMF	0.8906	0.8925	0.8824	0.8795	0.8612	0.8496
LNMF	0.9036	0.9258	0.9219	0.9214	0.9181	0.9229
GNMF	0.8824	0.8896	0.8925	0.8877	0.8752	0.8718
GDNMF	0.8925	0.8993	0.8747	0.8834	0.8689	0.8757

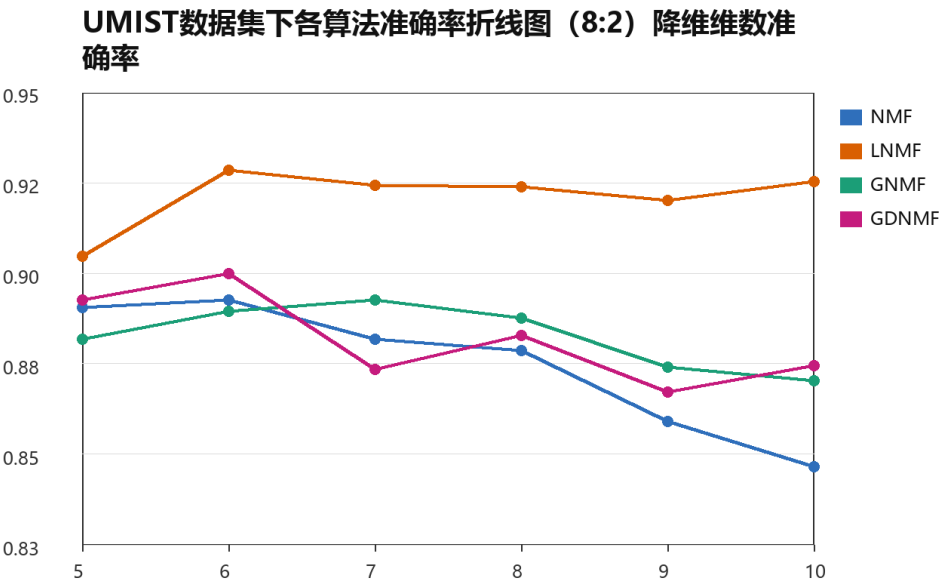


图 5.13 UMIST 数据集下各算法准确率折线图

表 5.6 在 UMIST 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 9：1）

维数算法	5	6	7	8	9	10
NMF	0.8957	0.9119	0.9003	0.8906	0.9018	0.8815
LNMF	0.9175	0.9448	0.9403	0.9311	0.9580	0.9327
GNMF	0.8937	0.9144	0.8997	0.9018	0.9180	0.8952
GDNMF	0.9033	0.9165	0.9008	0.8972	0.9124	0.8861

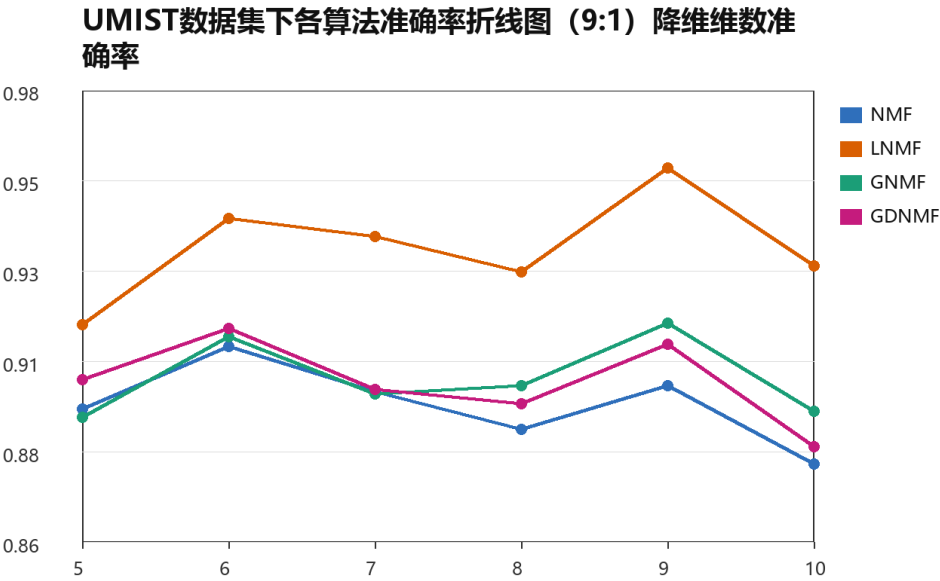


图 5.14 UMIST 数据集下各算法准确率折线图

5.3.3 Yale 数据集

经过迭代后，各算法进行人脸识别的准确率如下：

表 5.7 在 Yale 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 7：3）

维数算法	5	6	7	8	9	10
NMF	0.8000	0.8467	0.8833	0.9000	0.8767	0.8933
LNMF	0.7200	0.7533	0.7667	0.7767	0.7933	0.7967
GNMF	0.8000	0.8133	0.8733	0.8800	0.8867	0.9000
GDNMF	0.8133	0.8200	0.8567	0.8600	0.8767	0.8933

表 5.8 在 Yale 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 8：2）

维数算法	5	6	7	8	9	10
NMF	0.8356	0.8756	0.8756	0.8844	0.8889	0.8533
LNMF	0.7378	0.7511	0.7422	0.7556	0.7644	0.7556
GNMF	0.8311	0.8533	0.8756	0.8711	0.8578	0.8711
GDNMF	0.8533	0.8578	0.8489	0.8889	0.8711	0.8667

5.3.4 PIE 数据集

经过迭代后，各算法进行人脸识别的准确率如下：

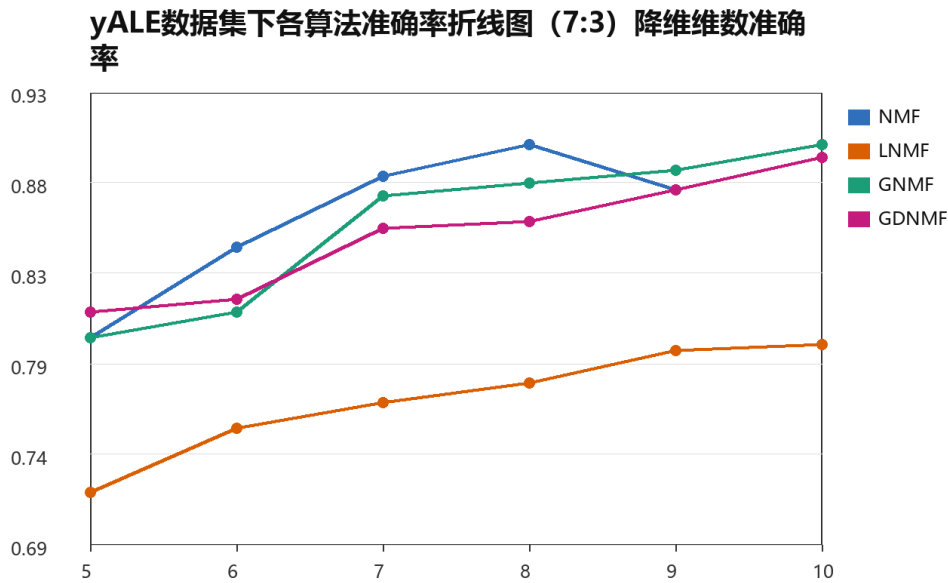


图 5.15 Yale 数据集下各算法准确率折线图

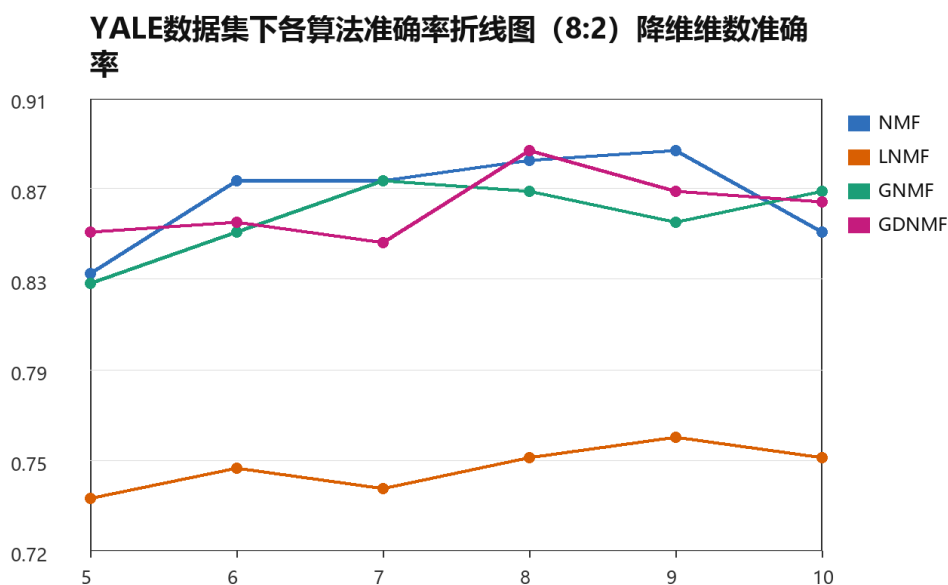


图 5.16 Yale 数据集下各算法准确率折线图

表 5.9 在 Yale 数据集下各算法进行人脸识别的准确率 (训练集: 测试集 = 9: 1)

维数算法	5	6	7	8	9	10
NMF	0.8667	0.8733	0.8733	0.8867	0.9267	0.9133
LNMf	0.7867	0.7533	0.7133	0.7933	0.7933	0.7533
GNMF	0.8267	0.8600	0.8467	0.8867	0.9000	0.8800
GDNMF	0.8667	0.8533	0.8467	0.9000	0.9000	0.9133

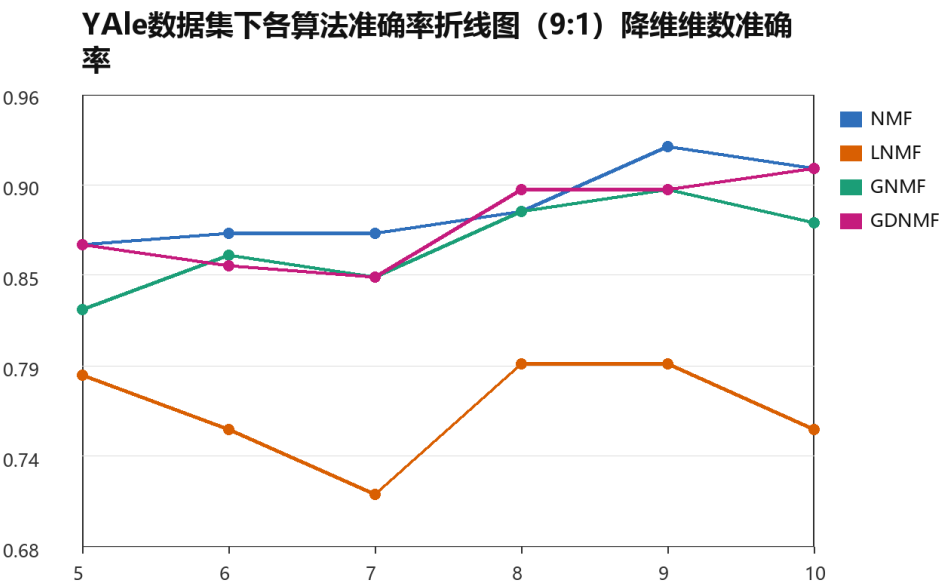


图 5.17 Yale 数据集下各算法准确率折线图

表 5.10 在 PIE 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 7：3）

维数算法	5	6	7	8	9	10
NMF	0.9933	0.9950	0.9977	1.0000	1.0000	1.0000
LNMF	0.7305	0.7916	0.8502	0.8840	0.8941	0.9050
GNMF	0.9903	0.9905	0.9968	0.9998	0.9992	1.0000
GDNMF	0.9912	0.9950	0.9968	0.9998	1.0000	0.9996

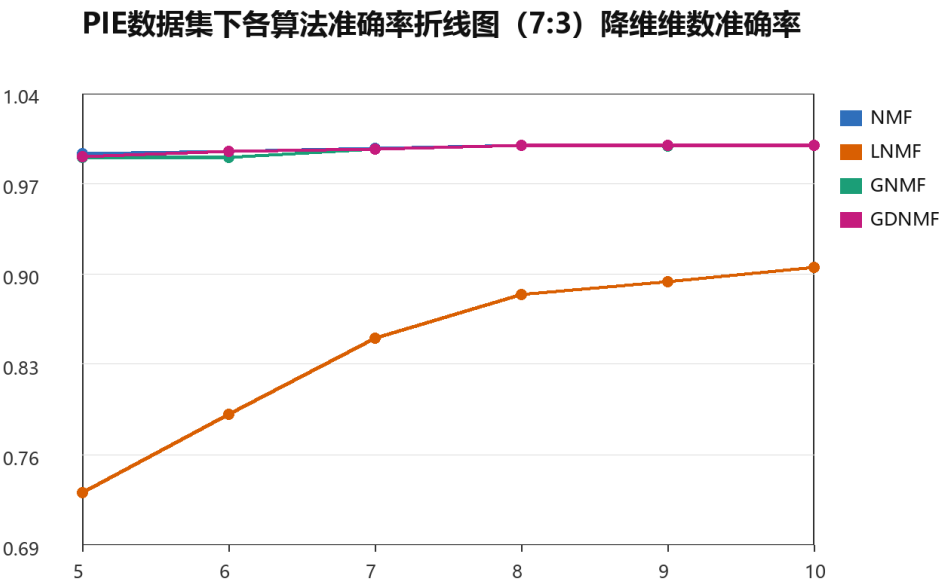


图 5.18 PIE 数据集下各算法准确率折线图

表 5.11 在 PIE 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 8：2）

维数算法	5	6	7	8	9	10
NMF	0.9943	0.9962	0.9986	1.0000	1.0000	1.0000
LNMF	0.7649	0.8310	0.8824	0.9048	0.9206	0.9448
GNMF	0.9914	0.9959	0.9982	0.9993	1.0000	1.0000
GDNMF	0.9955	0.9977	0.9995	1.0000	1.0000	1.0000

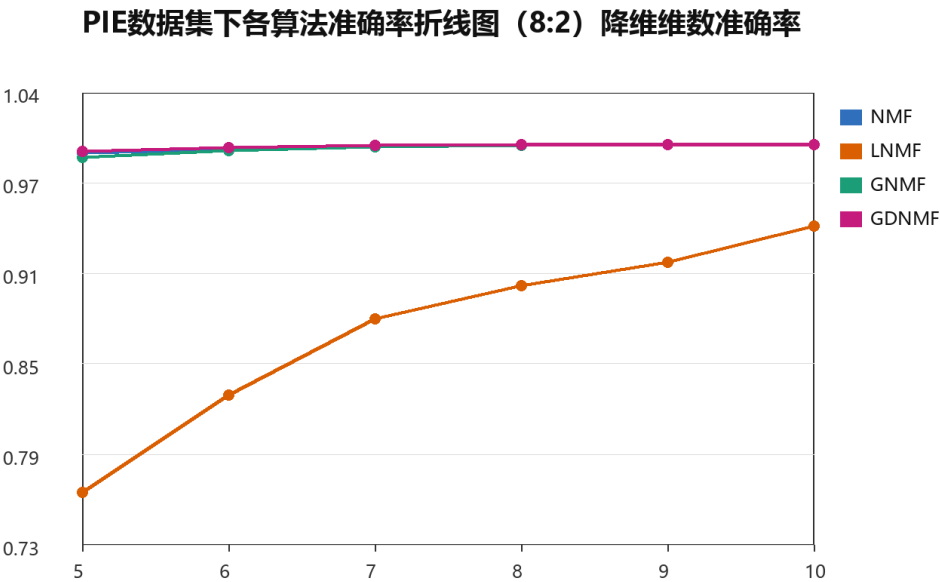


图 5.19 PIE 数据集下各算法准确率折线图

表 5.12 在 PIE 数据集下各算法进行人脸识别的准确率（训练集：测试集 = 9：1）

维数算法	5	6	7	8	9	10
NMF	0.9963	0.9983	1.0000	0.9995	1.0000	1.0000
LNMF	0.7831	0.8515	0.9029	0.9221	0.9419	0.9495
GNMF	0.9953	0.9973	0.9995	0.9995	1.0000	1.0000
GDNMF	0.9966	0.9998	0.9998	0.9998	0.9993	0.9998

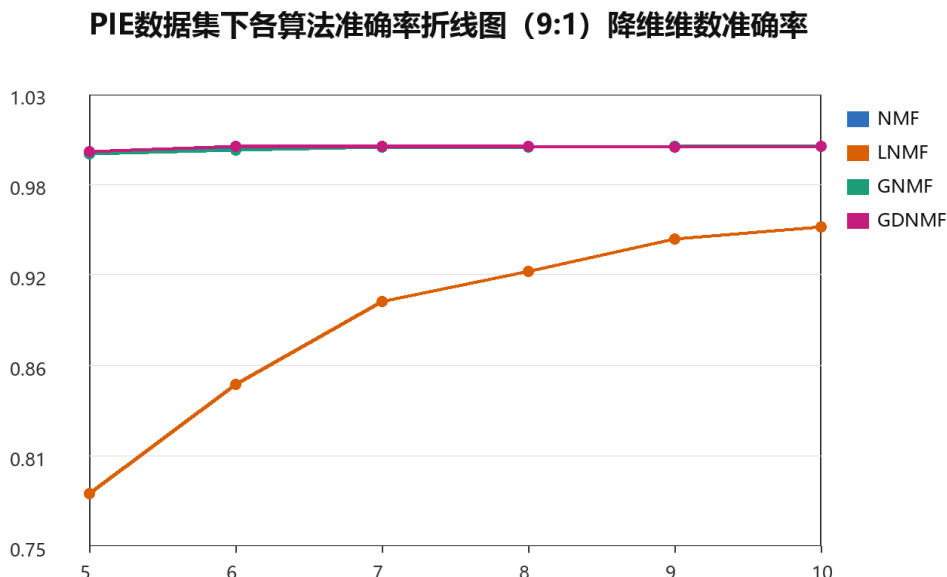


图 5.20 PIE 数据集下各算法准确率折线图

5.4 实验总结

在 5.3 小节开始的时候已经展示了在四个数据集上各算法的损失函数的收敛情况。由图易得，当迭代次数在 1000 时，损失函数的变化率趋于稳定，当迭代次数为 3000 时，变化率趋近于 0，此时可以认为算法收敛。所以本次实验分析将聚焦在训练集的占比和降维维数对于算法准确率的影响。

本次实验对于 4 个数据集都做了相同的对比，即在迭代次数为 3000 的情况下对训练集划分比例以及降维维数进行变化，以此进行对比实验。接下来将综合 4 个数据集的情况分别对两个变量进行分析，目的是找到对于使用非负矩阵分解进行人脸识别的最优配置。

关于训练集的划分，目前普遍的比例是训练集占总数的 80%。但是在本次实验中。训练集占比为 70% 的实验准确率相较于训练集占比 80% 的实验准确率较低，占比为 80% 的实验准确率相较于训练集占比 90% 的实验准确率较低，每个阶段准确率差距在 1% 左右。因为本次实验所选择的数据集普遍较小，并使用留出法进行随机划分，所以此结论并不具有普适性，但是仍可作为参考。

在 ORL 数据集上，当训练集划分比例为 7: 3 和 8: 2 时，降维维数为 7 时，各算法的准确率远超其他维数；当训练集划分比例为 9: 1 时，降维维数为 8 时相较其他各维数的准确率有明显优势。

在 UMIST 数据集上，当训练集划分比例为 7: 3 和 8: 2 时，准确率在 6 维后随着维数的增加而逐步下降，所以在维数为 6 时表现最好；当训练集划分比例为 9: 1 时，情况有所改变，在维数为 9 时出现大幅上升，但是在 10 维时又再次下降。虽然 9 维时准确率有所上升，但是

综合 4 种算法来看，还是不如 6 维时候的准确率。所以综合考虑，UMIST 数据集的维数取 6 最合适。当维数取 6 时，训练集比例为 9:1 时准确率最高，所以此组合为使用 UMIST 数据集进行非负矩阵分解的最优化配置。

在 Yale 数据集上，三种训练集划分比例的情况都类似，准确率较为平稳，变化率很小，准确率随维数增加缓慢增加。考虑到计算资源的有限性，所以取 10 维最好。当维数取 10 维的时候，三种训练集划分情况的准确率差距极小，但是当训练集比例为 9:1 时准确率最高，所以训练集比例为 9:1、降维维数为 10 维为使用 Yale 数据集进行非负矩阵分解的最优化配置。

在 PIE 数据集上的情况较为特殊，NMF、GNMF、GDNMF 算法的准确率在任何情况都接近 1，而 LNMF 算法在全部三种训练集划分情况下准确率随维数增加而增加，变化率在维数为 10 时趋于平稳。综合来看当维数取 10 时，训练集比例为 9:1 时准确率最高，所以此组合为使用 PIE 数据集进行非负矩阵分解的最优化配置。

比较各数据集，表现最好的为 ORL 以及 Yale 数据集，结果间有明显起伏，可以明确看出各维数间的优劣。情况最为特别的则是 PIE 数据集，由于准确率过高，近乎 100%，且主准确率曲线几乎毫无起伏。笔者在拿到数据后认为数据出现错误，进行了多次复测，但是结果相同。在仔细观察 PIE 数据集之后发现问题所在。PIE 数据集中同一类图片的主要变化为光照变化，拍摄角度为固定角度，所以图片的特征差异相对较小。在使用非负矩阵分解进行降维后，光照的大部分影响可以被剔除，所以准确率较高。而 LNMF 的优势是在于抓住图片的局部细节，所以在进行降维后准确率会比较不理想，但是也随着维数的增加而逐渐好转。

综上所述，每个数据集的情况都不尽相同，所以找到一个在任意数据集下都通用的最优化配置是难以实现的，这就要求我们实事求是，具体问题具体分析。

第六章 鲁棒性测试

6.1 必要性分析

在人脸识别实验中，使用非负矩阵分解算法时，进行鲁棒性测试是极为必要的。这是因为鲁棒性测试是全面评估算法性能的关键环节，尤其在面对复杂多变的实际应用场景时，算法的鲁棒性直接决定了其应用的有效性和可靠性^[31]。人脸识别技术广泛应用于安全监控、身份验证、人机交互等领域，这些领域要求算法能够在光照变化、遮挡、表情变化、姿态变化等多种不利条件下保持稳定和准确的性能。

通过鲁棒性测试，我们可以深入了解 NMF 算法在这些条件下的识别性能，从而更全面地评估其优缺点。此外，鲁棒性测试还能为算法的改进提供方向，帮助我们发现算法在哪些条件下性能较差，进而针对这些条件进行算法优化，提高其鲁棒性。在安全相关领域，算法的鲁棒性更是至关重要，因为它关系到系统的安全性和稳定性。因此，进行鲁棒性测试不仅是评估 NMF 算法性能的必要步骤，也是确保其在实际应用中能够发挥最大效能的关键环节。

6.2 实验过程

本次实验采用两种方式对数据集添加噪声，为椒盐噪声和高斯噪声。

椒盐噪声，也称为脉冲噪声，主要特点是在图像中随机出现黑白相间的亮暗点，这些点可能是亮的区域中的黑色像素，也可能是暗的区域中的白色像素。椒盐噪声的主要优点是它在视觉上比较直观，容易被人眼识别^[32]。然而，椒盐噪声的主要缺点是其对图像质量的影响较大，会严重影响图像的清晰度和后续的分析处理。在图像处理中，椒盐噪声通常需要通过中值滤波等非线性方法进行去除，但这种方法可能会使图像的边缘变得模糊^[33]。

高斯噪声则是指其概率密度函数服从高斯分布（即正态分布）的一类噪声^[34]。高斯噪声的主要优点是它在自然界中广泛存在，例如热噪声、散粒噪声等都属于高斯噪声。在图像处理中，高斯噪声的去除通常采用高斯滤波等线性平滑滤波方法^[35]，这些方法可以有效地降低噪声的影响，同时保持图像的清晰度。然而，高斯滤波也有其缺点，即在平滑噪声的同时可能会使图像的细节变得模糊^[36]。

总的来说，椒盐噪声和高斯噪声各有其优缺点。椒盐噪声在视觉上直观但影响大，而高斯噪声则广泛存在且易于通过线性方法去除，但在去除过程中可能会损失一些图像细节。在图像处理中，需要根据具体的噪声类型和图像特点选择合适的去噪方法。

以 ORL 数据库为例，经过添加噪声图片和原图片的对比如下：



图 6.1 原图



图 6.2 椒盐噪声



图 6.3 高斯噪声

6.3 结果分析

第四章的结果分析已经指出，四个数据库中效果最好的是 ORL 与 Yale 数据集，所以本次鲁棒性实验将主要关注这两个数据集。

针对收敛性的分析如下：

因为添加噪声总是只改变数据集中的图片，并对人脸识别成功率有所影响，但是并不会影响非负矩阵分解的降维过程，所以 4 种算法全都收敛。

由图易得，添加噪声对 NMF 算法的收敛性没有明显影响。

准确率分析如下：

由图可见，ORL 数据集对于噪声的抗干扰能力较强，而 Yale 数据集则容易受噪声影响。PIE 数据集在添加椒盐噪声后准确率的变化不大，但是在添加高斯噪声后准确率大幅下降，其原因还是在于 PIE 数据集的特殊性：同一类图片的最大不同是光照条件不同，而添加了高斯噪声后对图像的特征产生了较大的影响。各算法在不同噪声的影响下各有优劣，但是可以明显看出，LNMF 算法受噪声影响较大，在实际应用时需要针对数据集进行提前去噪。其他算法的抗噪声能力相似，噪声并不会对其准确率产生较大影响。但是 GDNMF 算法在特定情况下表现极差，所以在某些特定情况下并不适合使用。

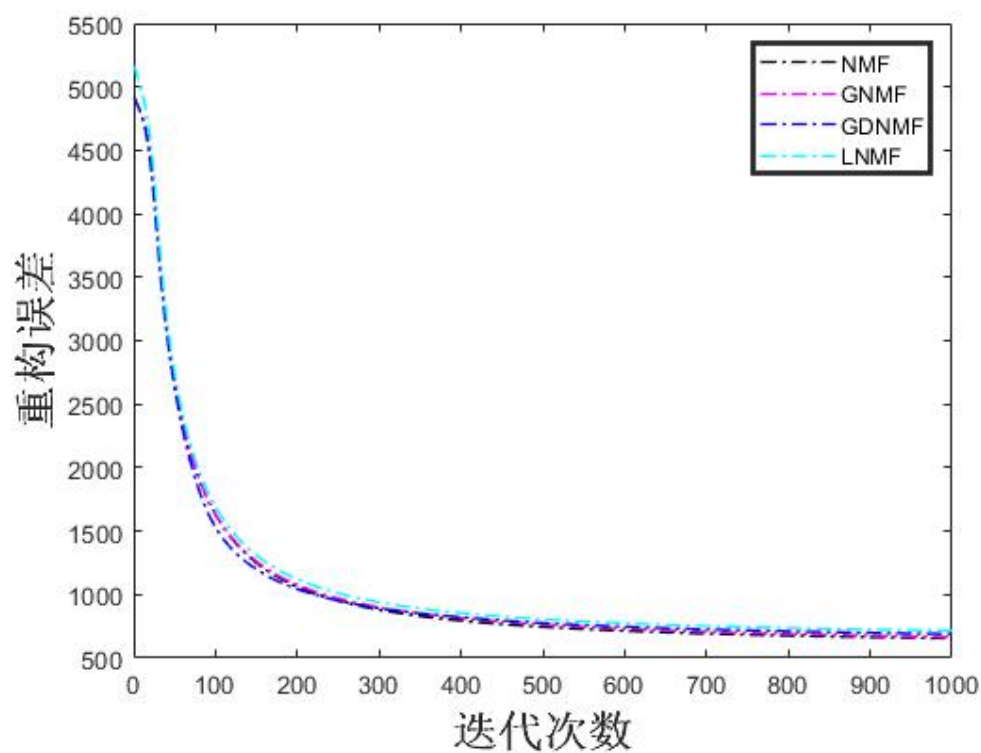


图 6.4 ORL 数据集添加椒盐噪声的收敛情况

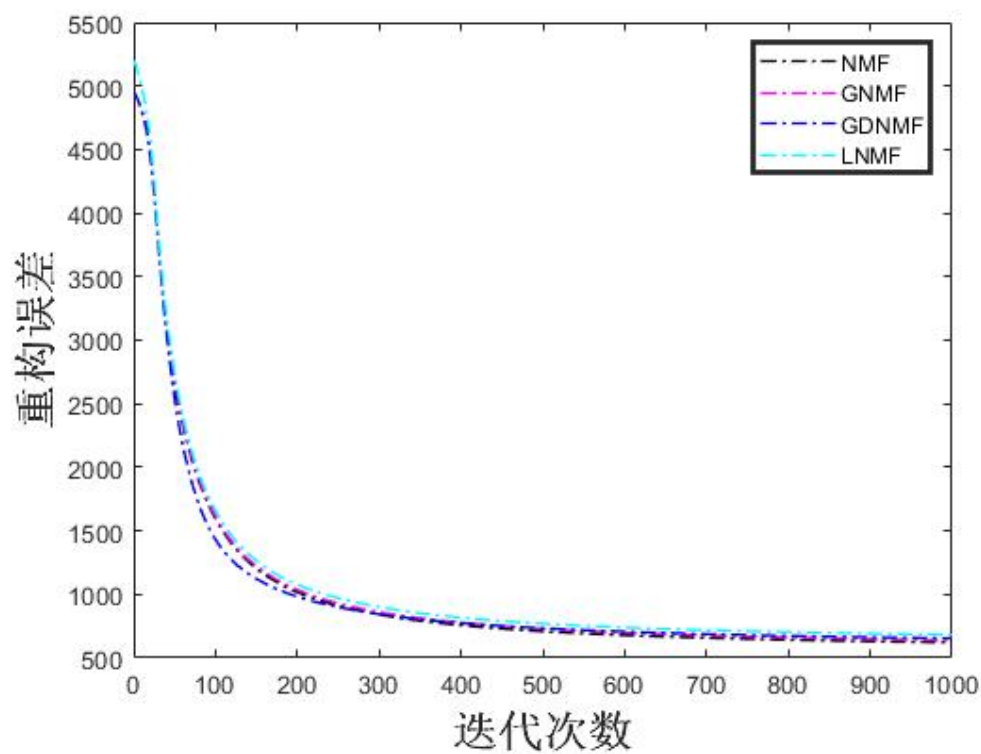


图 6.5 ORL 数据集添加高斯噪声的收敛情况

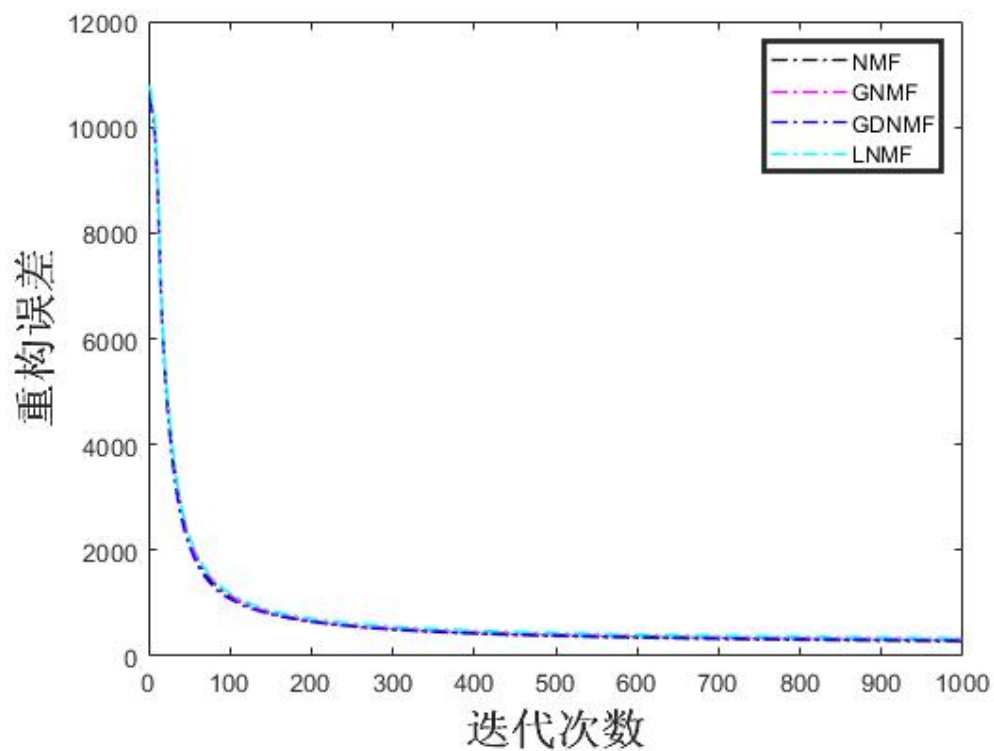


图 6.6 Yale 数据集添加椒盐噪声的收敛情况

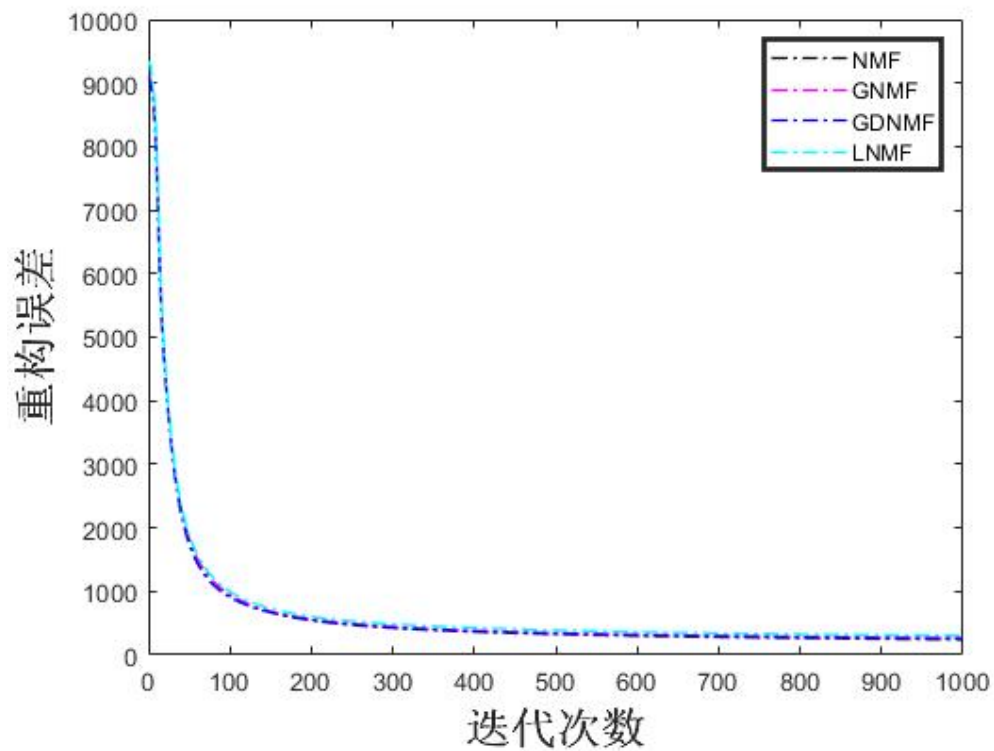


图 6.7 Yale 数据集添加高斯噪声的收敛情况

各算法在有噪条件下进行人脸识别的准确率的柱状图

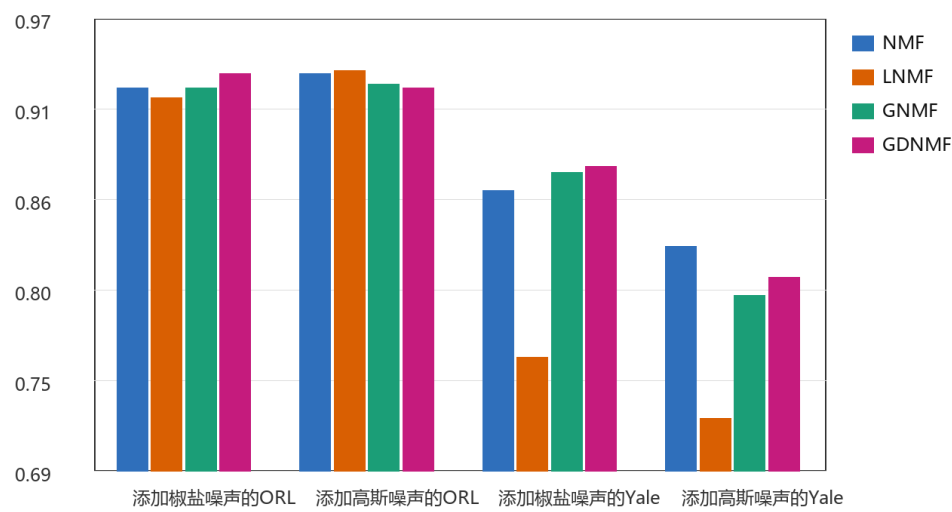


图 6.8 各算法在添加噪声后的 ORL 和 Yale 数据集下人脸识别准确率

各算法在有噪条件下进行人脸识别的准确率的柱状图

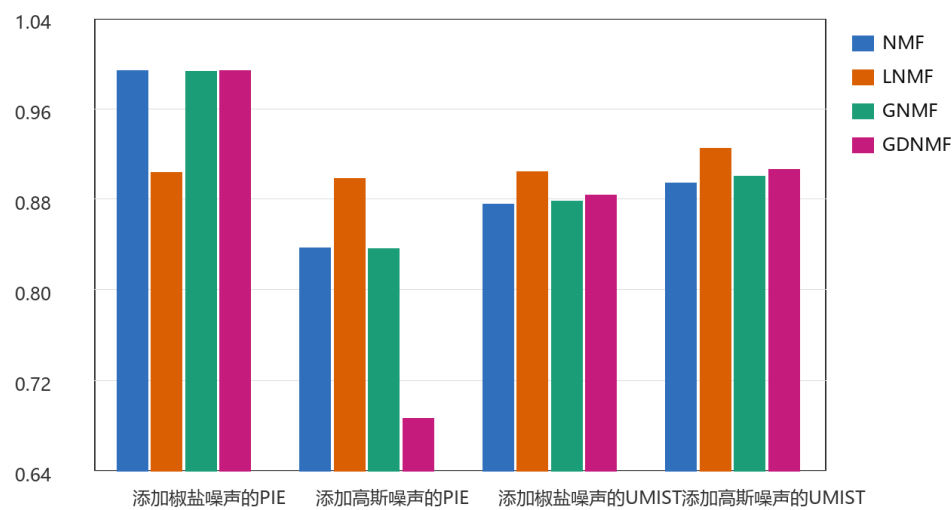


图 6.9 各算法在添加噪声后的 PIE 和 UMIST 数据集下人脸识别准确率

结论

NMF 算法，自诞生以来已走过二十多年的发展历程，但其影响力与活力却丝毫未减。随着时代的进步，科技的飞速发展，NMF 算法的变体算法如雨后春笋般不断涌现，它们以各自独特的方式对原算法进行了深度的完善与扩充，使其更加适应当前复杂多变的应用场景。经过一个多月的深入研究和实验，我逐渐理解了众多研究者对 NMF 算法持续投入热情的原因。这种热情，本质上源于 NMF 算法本身的卓越性，正是其强大的基础框架和无限的扩展潜力，才孕育出了如此丰富多彩的变体算法。在探索 NMF 算法收敛性的过程中，我更是深切地感受到了其深邃的数学魅力。一个核心定理，两个关键的引理，经过巧妙的变换、求导和线性处理，将线性代数的知识与思维发挥到了极致，最终推导出了 NMF 的经典迭代方式。这个过程中，与其说其作者是通过深入研究逐步取得成果，不如说是他们那天才般的跳脱思维在起着决定性的作用。NMF 算法，无疑是极简美学的杰出代表。它仅仅由一个损失函数和两个迭代公式构成，这就是它的全部。如果我们对算法有足够的理解和信心，甚至可以选择放弃损失函数，因为迭代公式本身就是由损失函数推导而来，迭代过程最终会导向收敛。回顾我参与的这个项目，我才意识到数据和结果的处理与存储占据了大量的时间和资源，而核心的 NMF 算法部分实际上只占据了非常小的篇幅。这种简洁性，正是 NMF 算法给予后来者无限创新空间的关键所在。尽管我在整个毕设过程中尽心尽力，但由于能力所限，我还是无法对 NMF 算法进行实质性的完善与改进。然而，这并不影响我对它未来发展的坚定信心。我相信，有着如此强大生命力和无限潜力的 NMF 算法，必将吸引更多优秀的研究者前来探索。不久的将来，一定会有新的变体算法被提出，NMF 算法仍将在科技领域持续闪耀，长盛不衰。

参考文献

- [1] Lee Daniel D., Seung H. Sebastian. Learning the parts of objects by non-negative matrix factorization[J]. Nature, 1999, 401(6755):788-791.
- [2] Long Xianzhong, Xiong Jian, Chen Lei. Robust automated graph regularized discriminative non-negative matrix factorization[J]. Multimedia Tools and Applications, 2021, 80(10):14867-14886.
- [3] Li S, Hou X, Zhang H, Cheng Q. Learning spatially localized, parts-based representation[C]. IEEE conference on computer vision and pattern recognition, 2002, pp 207–212.
- [4] Cai D, He X, Han J, Huang T. Graph regularized nonnegative matrix factorization for data representation[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2011, 33(8):1548–1560.
- [5] Long Xianzhong, Lu Hongtao, Peng Yong, Wenbin Li. Graph regularized discriminative non-negative matrix factorization for face recognition[J]. Multimedia Tools and Applications, 2014, 72(3): 2679-2699.
- [6] Aymen Bougacha, Jihene Boughariou, Ines Njeh, et al. A Rank-Two NMF Clustering: Application to glioblastomas characterization and comparative study[J]. Biomedical Engineering: Applications, Basis and Communications, 2019, 31(3).
- [7] Shu Zhenqiu, Zuo Furong, Wu Wenli, et al. Dual local learning regularized NMF with sparse and orthogonal constraints[J]. Applied Intelligence: The International Journal of Artificial Intelligence, Neural Networks, and Complex Problem-Solving Technologies, 2023, 53(7): 7713-7727.
- [8] Li Shang, Changxiong Zhou, Yunian Gu, et al. Face Recognition Using the Feature Fusion Technique Based on LNMF and NNSC Algorithms[C]. Advanced Intelligent Computing Theories and Applications. Springer, 2010:547-554.
- [9] Xiaoming Bai, Chengzhang Wang. LNMF Learning for Color Face Recognition[C]. The 2010 2nd International Conference on Future Computer and Communication. 2010:813-817.
- [10] Yang Zhou, Kai liu, Qingyu Dou, et al. LNMF: lightweight network for multi-focus image fusion[J]. Multimedia tools and applications, 2022, 81(16):22335-22353.
- [11] Shang Li, Zhou Yan, Chen Jie, et al. Image Denoising Using a Modified LNMF Algorithm[C]. International conference on computer science and service system, 2012:1840-1843.
- [12] Robust ECG biometrics using GNMF and sparse representation[J]. Pattern recognition letters, 2020,129(Jan.):70-76.
- [13] Yang Shuyuan, Zhang Xiantong, Yao Yigang, et al. Geometric Nonnegative Matrix Factorization (GNMF) for Hyperspectral Unmixing[J]. IEEE journal of selected topics in applied earth observations and remote sensing, 2015, 8(6):2696-2703.
- [14] Feng Gu, Wenju Zhang, Xiang Zhang, et al. GNMF Revisited: Joint Robust k-NN Graph and Reconstruction-Based Graph Regularization for Image Clustering[C]. International Conference on Artificial Neural Networks, 2017, 442-449.

- [15] Rafal Zdunek, Anh-huy Phan, Andrzej Cichocki. GNMF with Newton-Based Methods[C]. International Conference on Artificial Neural Networks, 2013:90-97.
- [16] Dai Xiangguang, Chen Guo, Li Chuandong. A discriminant graph nonnegative matrix factorization approach to computer vision[J]. Neural computing & applications, 2019, 31(11): 7879-7889.
- [17] Wan Minghua, Lai Zhihui, Ming Zhong, et al. An improve face representation and recognition method based on graph regularized non-negative matrix factorization[J]. Multimedia tools and applications, 2019,78(15):22109-22126.
- [18] Chen Chunchun, Zhu Wenjie, Peng Bo. Differentiated graph regularized non-negative matrix factorization for semi-supervised community detection[J]. Physica, A. Statistical mechanics and its applications, 2022, 604.
- [19] Li Huirong, Zhang Jianshe, Shi Guang, et al. Graph-based discriminative nonnegative matrix factorization with label information[J]. Neurocomputing, 2017, 266(Nov.29):91-100.
- [20] Xindong Wu, Vipin Kumar, J. ROSS QUINLAN, et al. Top 10 algorithms in data mining[J]. Knowledge and information systems, 2008, 14(1):1-37.
- [21] Du Mingjing, Ding Shifei, Jia Hongjie. Study on density peaks clustering based on k-nearest neighbors and principal component analysis[J]. Knowledge-based systems, 2016,99(May 1):135-145.
- [22] Zhang Shichao, Li Xuelong, Zong Ming, et al. Learning k for kNN Classification[J]. ACM transactions on intelligent systems and technology, 2017, 8(3).
- [23] D.H. Pandya, S.H. Upadhyay, S.P. Harsha. Fault diagnosis of rolling element bearing with intrinsic mode function of acoustic emission data using APF-KNN[J]. Expert Systems with Application, 2013, 40(10):4137-4145.
- [24] Cherkassky V, Ma Y. Practical selection of SVM parameters and noise estimation for SVM regression[J]. Neural Networks: The Official Journal of the International Neural Network Society, 2004, 17(1):113-126.
- [25] Suykens J.A.K., Vandewalle J. Least Squares Support Vector Machine Classifiers[J]. Neural Processing Letters, 1999, 9(3):293-300.
- [26] S. S. Keerthi, S. K. Shevade, C. Bhattacharyya. Improvements to Platt's SMO Algorithm for SVM Classifier Design[J]. Neural computation, 2001, 13(3):637-649.
- [27] Kyoung-jae Kim. Financial time series forecasting using support vector machines[J]. Neurocomputing, 2003, 55(1/2):307-319.
- [28] G. Ratsch, T. Onoda, K.-R. Muller. Soft Margins for AdaBoost[J]. Machine learning, 2001, 42(3):287-320.
- [29] Sun Ym, Kamel MS, Wong Akc, et al. Cost-sensitive boosting for classification of imbalanced data[J]. Pattern Recognition: The Journal of the Pattern Recognition Society, 2007, 40(12): 3358-3378.
- [30] Chan Jcw, Paelinckx D. Evaluation of Random Forest and Adaboost tree-based ensemble classification and spectral band selection for ecotope mapping using airborne hyperspectral imagery[J]. Remote Sensing of Environment: An Interdisciplinary Journal, 2008, 112(6):2999-3011.
- [31] Chan R.H., Chung-wa Ho, Nikolova M.. Salt-and-pepper noise removal by median-type noise detectors and detail-preserving regularization[J]. IEEE Transactions on Image Processing, 2005, 14(10):1479-1485.

- [32] Toh, K. K. V., Mat Isa, N. A.. Noise Adaptive Fuzzy Switching Median Filter for Salt-and-Pepper Noise Reduction[J]. IEEE signal processing letters, 2010,17(3):281-284.
- [33] Toh K.K.V., Ibrahim H., Mahyuddin M.N.. Salt-and-pepper noise detection and reduction using fuzzy switching median filter[J]. IEEE Transactions on Consumer Electronics, 2008, 54(4):1956-1961.
- [34] Ba-ngu Vo, Wing-kin Ma. The Gaussian Mixture Probability Hypothesis Density Filter[J]. IEEE Transactions on Signal Processing: A publication of the IEEE Signal Processing Society, 2006,54(11):4091-4104.
- [35] Fabrizio Russo. An image enhancement technique combining sharpening and noise reduction[J]. IEEE Transactions on Instrumentation and Measurement, 2002, 51(4):824-828.
- [36] Arasaratnam I., Haykin S., Elliott R. J.. Discrete-Time Nonlinear Filtering Algorithms Using Gauss–Hermite Quadrature[J]. Proceedings of the IEEE, 2007, 95(5):953-977.

致谢

在此，我衷心地感谢所有在我撰写这篇毕业论文过程中给予我指导、支持和帮助的人。首先，我要向我的导师龙显忠老师致以最深的谢意。老师严谨的学术态度、深厚的专业知识以及敏锐的洞察力，在我论文的选题、框架构建、实验设计以及撰写过程中，都给予了我无微不至的指导和帮助。每当我遇到困难时，老师总是能够耐心解答，给予我宝贵的建议，使我的研究能够顺利进行。老师的教诲不仅让我在学术上有所收获，更在人生道路上受益匪浅。其次，我要感谢我的同学们。在论文的撰写过程中，我们共同探讨、互相学习，分享彼此的想法和见解。他们的支持和帮助，让我在面对困难时不再孤单，也让我更加深入地理解了计算机领域的各种技术和理论。同时，我也要感谢我的家人。他们一直是我坚实的后盾，给予我无私的爱和关心。在我撰写论文期间，他们不仅为我提供了良好的生活环境，还时常鼓励我、支持我，让我能够专注于学术研究。此外，我还要感谢所有参考文献的作者们。他们的研究成果为我提供了宝贵的参考和借鉴，使我的论文能够更加深入、全面地探讨非负矩阵分解领域的相关问题。最后，我要感谢学校为我提供的良好的学术氛围和丰富的学术资源。这些资源不仅让我能够接触到最前沿的学术成果，还使我的学术视野更加开阔。再次感谢所有在我论文撰写过程中给予我帮助和支持的人。你们的关心和帮助是我前进的动力，也是我不断追求卓越的动力源泉。我将永远铭记在心，并以此为动力，继续努力学习和研究，为计算机领域的发展贡献自己的力量。